UNIVERSIDAD RAFAEL LANDIVAR

FACULTAD DE INGENIERIA

INTELIGENCIA ARTIFICIAL - SECCION 01

PROYECTO 1

CLASIFICACION DE SOLICITUDES A MESA DE AYUDA

Clasificacion de Texto usando Naive Bayes Multinomial

Integrantes:

Jose Mendizabal - Carne 1077222

Luis Fernando Samayoa Sagastume - Carne 1227421

Catedratico:

Ing. Max Cerna

Fecha de entrega:

Guatemala, Primer Semestre 2026

Indice

1. Introduccion 3

2. Definicion del Problema 4

3. Objetivos 5

3.1. Objetivo General 5

3.2. Objetivos Especificos 5

4. Descripcion del Dataset 6

5. Tecnologias Utilizadas 8

6. Arquitectura de la Solucion 11

7. Preprocesamiento de Texto 13

8. Algoritmo Naive Bayes - Explicacion Matematica 17

9. Metricas de Evaluacion 22

10. K-Folds Cross Validation 25

11. Diagrama de Casos de Uso 27

12. Diagrama de Flujo General 28

13. Diagrama de Componentes 29

14. Diagrama de Secuencias 30

15. Resultados Obtenidos 31

16. Analisis Critico 35

17. Evidencias de Funcionamiento 38

18. Conclusiones y Aprendizajes 40

19. Referencias 42

1. Introduccion

En la era digital actual, las empresas que ofrecen servicios al cliente reciben diariamente grandes volumenes de solicitudes de soporte tecnico. Estas solicitudes llegan por multiples canales (correos electronicos, formularios web, chats en vivo) y deben ser dirigidas al departamento correcto para obtener una resolucion oportuna. El proceso tradicional de clasificacion manual presenta varios inconvenientes: consume tiempo valioso del personal, introduce errores humanos, y escala mal cuando el volumen de solicitudes crece. Este proyecto aborda precisamente esta problematica mediante la construccion de un sistema de clasificacion automatica basado en tecnicas clasicas de Inteligencia Artificial.

El objetivo central del trabajo consiste en implementar desde cero un clasificador de texto basado en el algoritmo Naive Bayes Multinomial, sin recurrir a bibliotecas especializadas en Machine Learning como scikit-learn, TensorFlow o PyTorch. Esta restriccion, lejos de ser un obstaculo, constituye el principal valor pedagogico del proyecto: obliga al estudiante a comprender en profundidad cada etapa del proceso, desde la limpieza del texto hasta la inferencia probabilistica, pasando por el suavizado de Laplace y el uso de logaritmos para evitar el underflow numerico.

El sistema desarrollado se compone de tres capas claramente diferenciadas: un motor de inferencia implementado en Python que encapsula toda la logica del clasificador, un servidor web construido con Flask que expone el modelo a traves de una API REST, y una interfaz grafica responsiva desarrollada con HTML, CSS y JavaScript que simula un portal de tickets real. La integracion de estas tres capas permite al usuario final redactar una solicitud de soporte y recibir, en tiempo real, la clasificacion automatica de su ticket en alguna de las once categorias disponibles (ACCOUNT, BILLING, CANCEL, CONTACT, DELIVERY, FEEDBACK, INVOICE, ORDER, PAYMENT, REFUND, SHIPPING, SUBSCRIPTION).

El dataset utilizado es el Bitext Customer Support LLM Chatbot Training Dataset, un corpus de 26,872 solicitudes etiquetadas con coherencia semantica, disponible publicamente en Hugging Face. La evaluacion del modelo se realiza mediante validacion cruzada K-Folds con K=5, implementada manualmente, y se reportan las metricas

estandar en tareas de clasificacion multiclase: precision, recall, F1-score por clase, accuracy global, macro F1 y matriz de confusion. Los resultados obtenidos superan el 99% de accuracy promedio con una varianza extremadamente baja entre folds, lo que demuestra la robustez del modelo y la calidad del preprocesamiento aplicado.

Este documento presenta, en primer lugar, la definicion formal del problema y los objetivos del proyecto. Luego describe detalladamente el dataset utilizado y las tecnologias elegidas, justificando cada decision tecnica. A continuacion se explica la arquitectura general del sistema y se profundiza en la matematica subyacente al algoritmo Naive Bayes. Las secciones finales presentan los resultados experimentales obtenidos, un analisis critico de las fortalezas y limitaciones del modelo, evidencias visuales del funcionamiento de la aplicacion web, y las conclusiones extraidas a lo largo del desarrollo del proyecto.

2. Definicion del Problema

Las empresas modernas que operan plataformas de comercio electronico, servicios de suscripcion o plataformas SaaS reciben diariamente cientos o miles de solicitudes de soporte al cliente. Estas solicitudes, redactadas en lenguaje natural por usuarios con distintos niveles de formalidad y precision, deben ser enrutadas al departamento correspondiente: el equipo de facturacion, el equipo de envios, el equipo de cuentas, entre otros. Cada departamento tiene especialistas entrenados para resolver problemas especificos, por lo que el enrutamiento incorrecto de un ticket genera demoras, transferencias entre areas, frustracion del cliente y, en ultimo termino, perdida de ingresos o de fidelidad.

Historicamente, esta tarea de clasificacion ha sido realizada de forma manual por operadores humanos. Un agente de primer nivel lee la solicitud, identifica la categoria a la que pertenece y la reasigna al equipo correcto. Este proceso, ademas de consumir tiempo valioso que podria dedicarse a la resolucion efectiva de problemas, es propenso a errores, especialmente cuando el volumen de tickets es alto o cuando los agentes son nuevos y aun no dominan todas las categorias. La variabilidad entre agentes tambien genera inconsistencias: un mismo tipo de ticket puede ser clasificado de forma distinta por dos agentes diferentes.

El problema concreto que este proyecto busca resolver es el siguiente: dada una solicitud de soporte al cliente escrita en ingles, en formato de texto libre, con longitud variable y sin ninguna estructura predefinida, el sistema debe ser capaz de asignar automaticamente una unica categoria tematica entre un conjunto finito de once clases predefinidas. La decision debe tomarse en tiempo real (idealmente en menos de 100 milisegundos), y debe estar acompanada de una medida de confianza que permita al operador humano decidir si aceptar la clasificacion automatica o revisarla manualmente en casos ambiguos.

Adicionalmente, el sistema debe cumplir con varios requisitos funcionales y no funcionales. Entre los requisitos funcionales se encuentran: la capacidad de ser entrenado con un dataset de tamano mediano (mayor a 8,000 instancias), la posibilidad de guardar y cargar el modelo entrenado para evitar reentrenamientos innecesarios, y la

existencia de una interfaz web que simule un portal de tickets real. Entre los requisitos no funcionales destacan: la implementacion manual del algoritmo sin recurrir a bibliotecas de Machine Learning de alto nivel, el uso de logaritmos para garantizar la estabilidad numerica, y la aplicacion de validacion cruzada K-Folds para una evaluacion estadisticamente robusta.

El desafio tecnico principal radica en que el texto natural es inherentemente ruidoso y variable: los usuarios escriben con errores tipograficos, utilizan abreviaturas, insertan numeros de orden, direcciones de correo electronico, URLs, y usan distintas formas morfologicas de una misma palabra (plurales, conjugaciones verbales). El modelo debe ser robusto ante toda esta variabilidad y aun asi encontrar los patrones lexicos distintivos de cada categoria. Ademas, al ser un problema multiclase con once categorias y un dataset desbalanceado (algunas clases tienen el doble o triple de ejemplos que otras), es fundamental utilizar metricas de evaluacion adecuadas que no queden dominadas por la clase mayoritaria.

3. Objetivos

3.1 Objetivo General

Disenar, implementar y evaluar un sistema de clasificacion automatica de solicitudes de soporte al cliente, basado en el algoritmo Naive Bayes Multinomial desarrollado desde cero en Python, integrado con una aplicacion web funcional que simule un portal de tickets real, capaz de asignar categorias de forma precisa y en tiempo real a partir del contenido textual de las solicitudes.

3.2 Objetivos Especificos

- **Preprocesar** los datos textuales del dataset aplicando tecnicas de limpieza con expresiones regulares, tokenizacion, eliminacion de palabras vacias (stopwords) y reduccion morfologica mediante stemming, para obtener una representacion limpia y consistente de cada documento.
- **Construir manualmente un vocabulario global** mediante el enfoque Bag of Words a partir del corpus de entrenamiento, registrando la frecuencia con la que cada palabra aparece en cada clase.
- **Implementar el algoritmo Naive Bayes Multinomial** sin recurrir a bibliotecas de Machine Learning de alto nivel, incluyendo el calculo de probabilidades a priori por clase, el calculo de verosimilitudes con suavizado de Laplace para manejar palabras no vistas, y el uso de sumas de logaritmos para evitar el problema de underflow numerico durante la inferencia.
- **Evaluar el modelo** mediante la tecnica de validacion cruzada K-Folds con K=5, implementada manualmente, con el fin de obtener metricas robustas y menos dependientes de un unico particionado aleatorio del dataset.
- **Analizar e interpretar** las metricas de evaluacion obtenidas, incluyendo precision, recall y F1-score por clase, accuracy global, macro F1, y la matriz de confusion, identificando que clases se predicen con mayor facilidad y cuales generan mas confusiones entre si.
- **Serializar y persistir** el modelo entrenado en un archivo binario utilizando la biblioteca pickle de Python, de modo que pueda ser cargado posteriormente sin necesidad de reentrenamiento.

- **Desarrollar una interfaz web funcional** utilizando HTML, CSS y JavaScript con el framework Bootstrap 5, que simule un sistema real de tickets con campos para identificador autogenerado, asunto y descripcion, mostrando en tiempo real la categoria predicha y las probabilidades por clase.
- **Integrar el backend y el frontend** a traves de un servidor Flask que exponga endpoints REST, permitiendo la comunicacion asincrona entre la pagina web y el motor de inferencia mediante llamadas fetch() y respuestas en formato JSON.

4. Descripcion del Dataset

La calidad y caracteristicas del dataset son factores determinantes en el rendimiento final de cualquier sistema de aprendizaje supervisado. Para este proyecto se utilizo el Bitext Customer Support LLM Chatbot Training Dataset, disponible publicamente en la plataforma Hugging Face bajo la identificacion bitext/Bitext-customer-support-llm-chatbot-training-dataset. Se trata de un corpus especificamente curado para tareas de clasificacion de intenciones en sistemas de soporte al cliente, ampliamente utilizado en la literatura academica y en aplicaciones comerciales de chatbots.

4.1 Caracteristicas Generales

El dataset contiene un total de 26,872 instancias, cada una representando una solicitud individual de soporte redactada en ingles. Cada instancia esta compuesta por dos campos principales que resultan relevantes para el proyecto: el campo instruction, que contiene el texto de la solicitud tal como fue redactada, y el campo category, que contiene la etiqueta de clase asignada. La clasificacion abarca un total de once categorias semanticas distintas, cada una asociada a un tipo especifico de solicitud: ACCOUNT (cuenta de usuario), CANCEL (cancelacion), CONTACT (contacto con soporte), DELIVERY (entrega), FEEDBACK (retroalimentacion), INVOICE (factura), ORDER (pedido), PAYMENT (pago), REFUND (reembolso), SHIPPING (envio) y SUBSCRIPTION (suscripcion).

4.2 Justificacion de la Eleccion del Dataset

Originalmente, el enunciado del proyecto proponia como dataset principal uno alojado en Kaggle bajo el usuario suraj520. Sin embargo, tras realizar un analisis exploratorio inicial, se detecto que las etiquetas de categoria en ese dataset habian sido asignadas de forma aleatoria respecto al contenido real del texto. Esto significa que un mismo ticket sobre un reembolso podia aparecer etiquetado como ORDER o como BILLING sin ninguna coherencia semantica, lo cual hace imposible que cualquier algoritmo de aprendizaje automatico pueda identificar patrones reales. El catedratico autorizo en consecuencia el uso del dataset alternativo de Bitext.

El dataset de Bitext presenta varias ventajas fundamentales frente a la opcion original. En primer lugar, sus etiquetas fueron generadas mediante un proceso controlado y verificado, de modo que cada ejemplo es semanticamente coherente con su categoria asignada. En segundo lugar, cuenta con un volumen considerablemente mayor (26,872 instancias frente a apenas unos pocos miles), lo que proporciona mas evidencia para el entrenamiento del modelo. En tercer lugar, las once categorias cubren practicamente todos los escenarios tipicos de un centro de soporte al cliente de comercio electronico, lo que da al proyecto un caracter realista. Finalmente, el dataset ha sido utilizado previamente en publicaciones academicas y productos comerciales, lo que da una referencia externa sobre los resultados esperables.

4.3 Distribucion de las Clases

Un aspecto critico del dataset es su distribucion de clases, la cual no es uniforme. La siguiente figura muestra la cantidad de ejemplos disponibles para cada una de las once categorias:

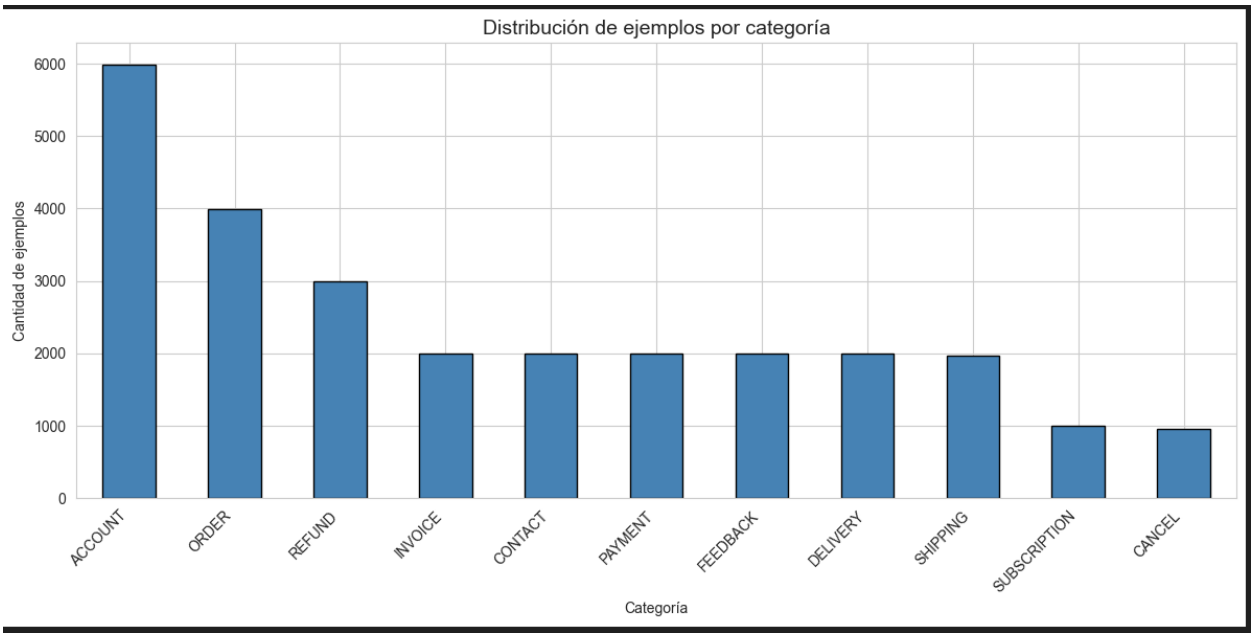

Figura 1. Distribucion de ejemplos por categoria en el dataset Bitext.

Como puede observarse en la figura anterior, la clase ACCOUNT es la mas representada con aproximadamente 6,000 ejemplos, seguida por ORDER con aproximadamente 4,000 y REFUND con 3,000. Las clases intermedias (CONTACT, DELIVERY, FEEDBACK,

INVOICE, PAYMENT, SHIPPING) tienen alrededor de 2,000 ejemplos cada una, mientras que las clases menos representadas son CANCEL y SUBSCRIPTION con aproximadamente 1,000 ejemplos cada una. Esta distribucion desbalanceada es una caracteristica importante que debe tenerse en cuenta al interpretar las metricas finales del modelo, ya que clasificadores como Naive Bayes tienden a favorecer a la clase mayoritaria debido al peso de la probabilidad a priori en el calculo de la verosimilitud posterior.

4.4 Caracteristicas del Texto

El analisis exploratorio revelo que los textos del dataset son relativamente cortos, con una longitud promedio de aproximadamente 50 a 150 caracteres y entre 8 y 25 palabras por solicitud. Esta brevedad es consistente con la naturaleza real de las solicitudes de soporte, donde los usuarios tienden a describir su problema de forma concisa. Una caracteristica particular del dataset son los placeholders: textos entre dobles llaves (por ejemplo, {{Order Number}} o {{Account Type}}) que representan datos que serian completados automaticamente en un sistema real. Estos placeholders deben eliminarse durante el preprocesamiento, ya que no contienen informacion semantica y distorsionan el vocabulario.

No se encontraron valores nulos ni filas duplicadas en el dataset, lo que indica que se trata de un corpus bien curado y listo para ser utilizado directamente en tareas de clasificacion sin requerir procesos adicionales de limpieza a nivel de registro. El idioma de todo el corpus es ingles, por lo que las decisiones tomadas en el preprocesamiento (stopwords, stemming) se aplicaron en ese idioma.

5. Tecnologias Utilizadas

La eleccion de tecnologias para un proyecto de Inteligencia Artificial con interfaz web debe equilibrar varios factores: la madurez y estabilidad de las herramientas, su idoneidad para la tarea especifica, las restricciones del proyecto (en este caso, la prohibicion de bibliotecas de Machine Learning de alto nivel), y la facilidad de desarrollo y mantenimiento. A continuacion se justifica detalladamente cada una de las tecnologias incorporadas en el proyecto, separadas por capa arquitectonica.

5.1 Lenguaje Principal: Python 3.10+

Se eligio Python como lenguaje principal del proyecto por multiples razones. En primer lugar, Python es el estandar de facto en el ambito de la Inteligencia Artificial y el procesamiento de lenguaje natural, lo que significa que existe una vasta cantidad de documentacion, ejemplos y comunidad que respaldan cualquier decision tecnica. En segundo lugar, su sintaxis clara y expresiva permite centrar la atencion en la logica del algoritmo en lugar de en detalles del lenguaje, lo cual es especialmente valioso cuando el objetivo pedagogico es comprender un algoritmo desde cero. En tercer lugar, Python cuenta con estructuras de datos nativas (diccionarios, conjuntos, listas por comprension) que resultan ideales para manipular vocabularios, conteos de frecuencia y probabilidades sin necesidad de bibliotecas externas.

Se eligio especificamente la version 3.10 o superior para aprovechar mejoras de rendimiento, sintaxis mas clara en manejo de errores (por ejemplo, la sintaxis de pattern matching) y compatibilidad con las ultimas versiones de las bibliotecas auxiliares utilizadas. Las versiones anteriores de Python (como 3.7 y 3.8) ya no reciben soporte extendido, por lo que utilizar una version moderna garantiza tambien actualizaciones de seguridad.

5.2 Bibliotecas para el Motor de Inferencia

5.2.1 NLTK (Natural Language Toolkit)

NLTK es una biblioteca madura para procesamiento de lenguaje natural en Python, desarrollada inicialmente en la Universidad de Pensilvania. Se utilizo unicamente para

dos tareas puntuales que el enunciado del proyecto permite explicitamente: la tokenizacion de texto (funcion word_tokenize) y la obtencion de listas de stopwords en ingles (corpus stopwords.words('english')). Adicionalmente se empleo el algoritmo de stemming de Porter (PorterStemmer), un algoritmo clasico desarrollado por Martin Porter en 1980, que reduce palabras a su raiz morfologica mediante un conjunto de reglas de transformacion.

La decision de usar NLTK en lugar de implementar la tokenizacion manualmente se fundamenta en que el tokenizador de NLTK maneja correctamente numerosos casos borde (contracciones, signos de puntuacion pegados a palabras, abreviaturas) que una implementacion con str.split() manejaria mal, introduciendo ruido innecesario. Sin embargo, NLTK no interviene en ninguna parte del algoritmo de clasificacion en si: el Bag of Words, las probabilidades, el suavizado y la inferencia se calculan con Python puro.

5.2.2 Pickle

Pickle es la biblioteca estandar de Python para serializacion de objetos. Se utilizo para persistir el modelo entrenado en un archivo binario (modelo_entrenado.pkl) junto con el preprocesador y los metadatos asociados. La ventaja principal de pickle es que permite guardar y cargar objetos Python arbitrarios sin necesidad de escribir codigo de serializacion personalizado; basta con invocar pickle.dump() y pickle.load(). Esto encaja perfectamente con los requisitos del proyecto, que exigen la persistencia del modelo entrenado en un archivo cargable. Aunque se podria haber usado JSON, el formato pickle permite preservar la estructura exacta de clases y conjuntos de Python (como el conjunto del vocabulario), algo que JSON no soporta nativamente.

5.2.3 Pandas y NumPy

Pandas se utilizo exclusivamente para la carga del dataset en formato CSV (funcion read_csv) y para facilitar la exploracion inicial del corpus. No se utilizo en ningun paso del algoritmo de clasificacion. NumPy, de forma similar, se empleo unicamente en la etapa de visualizacion para manipular la matriz de confusion como un arreglo bidimensional antes de graficarla. La decision de no incorporar estas bibliotecas al nucleo del clasificador obedece al requisito del proyecto: el algoritmo Naive Bayes se

implementa con estructuras nativas de Python (diccionarios, Counter, defaultdict) para demostrar su entendimiento profundo.

5.2.4 Matplotlib y Seaborn

Estas bibliotecas se utilizaron exclusivamente para generar visualizaciones (graficos de barras, heatmaps) que se incluyen en este informe. Matplotlib proporciona la base sobre la que se construyen los graficos, mientras que Seaborn anade una capa de estilos mas modernos y funciones especializadas para graficos estadisticos como heatmaps. La matriz de confusion, en particular, se graficó con Seaborn dada la facilidad con la que esta biblioteca genera heatmaps anotados con las frecuencias absolutas y las proporciones por fila. Al igual que pandas y numpy, estas bibliotecas no intervienen en el algoritmo de clasificacion.

5.2.5 Hugging Face datasets

La biblioteca datasets de Hugging Face se utilizo unicamente para la descarga inicial del corpus de Bitext. Esta biblioteca permite acceder a miles de datasets publicos con una sola linea de codigo, manejando automaticamente la descarga, el caching y la conversion a estructuras de datos comunes. Su uso esta completamente aislado a un unico script (descargar_dataset.py) que genera un archivo CSV local, y no forma parte del flujo de entrenamiento ni de inferencia del modelo.

5.3 Tecnologias del Servidor Web (Backend)

5.3.1 Flask 3.0

Flask es un micro-framework web en Python desarrollado por Armin Ronacher. Se eligio por encima de alternativas mas pesadas (como Django) debido a varias razones concretas. Primero, Flask sigue la filosofia de 'micro' framework: incluye solo lo esencial (ruteo, manejo de requests y responses, templates) y permite al desarrollador anadir solo las piezas que necesita. Para un proyecto academico con una API simple y una unica pagina web, esta simplicidad es una ventaja clara frente a frameworks mas estructurados que imponen convenciones innecesarias.

Segundo, Flask permite arrancar un servidor web funcional en menos de diez lineas de codigo, lo cual agiliza enormemente el desarrollo y la experimentacion. Tercero, Flask se

integra de forma natural con bibliotecas de Python como pickle, NLTK y cualquier codigo de clasificacion personalizado, ya que todo esta en el mismo entorno de Python sin necesidad de comunicacion inter-proceso. Cuarto, Flask cuenta con un sistema de templates basado en Jinja2 que permite servir paginas HTML dinamicas de forma sencilla. Finalmente, la version 3.0 de Flask, liberada recientemente, incorpora mejoras en el manejo de rutas y en la serializacion JSON que resultan utiles para la API del proyecto.

5.4 Tecnologias del Frontend

5.4.1 HTML5, CSS3 y JavaScript (Vanilla)

El frontend se construyo utilizando las tecnologias web estandar: HTML5 para la estructura del documento, CSS3 para los estilos, y JavaScript vanilla (sin frameworks como React, Angular o Vue) para la logica interactiva. La decision de no utilizar un framework de JavaScript responde a la escala del proyecto: la interfaz consiste en un unico formulario con resultados renderizados dinamicamente, un caso que JavaScript vanilla maneja perfectamente bien sin anadir la complejidad y el peso de un framework completo. Ademas, esta decision mantiene el codigo mas comprensible y educativo, permitiendo apreciar como funcionan las peticiones fetch(), la manipulacion del DOM y los event listeners sin capas de abstraccion adicionales.

5.4.2 Bootstrap 5

Bootstrap 5 es un framework CSS desarrollado originalmente por Twitter, que proporciona un conjunto predefinido de componentes visuales (botones, formularios, tarjetas, alertas, tipografias) y un sistema de rejilla responsivo basado en flexbox. Se utilizo en el proyecto para darle al frontend un aspecto profesional y moderno con un esfuerzo de diseno minimo. Bootstrap 5 representa una evolucion significativa respecto a versiones anteriores: elimino la dependencia de jQuery, redujo el tamano del bundle, mejoro la accesibilidad y adopto CSS moderno como las variables CSS y flexbox.

El framework se incluye directamente desde un CDN (Content Delivery Network), lo que evita la necesidad de instalar dependencias npm y facilita el despliegue. Adicionalmente, se utilizo Font Awesome 6 para los iconos vectoriales, lo cual anade pequenos detalles

visuales (iconos de sobre, tickets, graficos) que mejoran significativamente la experiencia del usuario.

5.5 Herramientas de Desarrollo

El desarrollo se realizo utilizando Visual Studio Code como editor principal, con Jupyter Notebook para la fase de exploracion inicial del dataset. El control de versiones se gestiono con Git y GitHub Desktop, manteniendo un repositorio publico donde se registran todos los commits del proyecto. Los entornos virtuales se gestionaron con venv (la herramienta estandar incluida en Python) para aislar las dependencias del proyecto del resto del sistema, una practica recomendada que evita conflictos entre proyectos y facilita la reproducibilidad. Las dependencias se documentan en el archivo requirements.txt, el cual permite instalar todo el entorno con un unico comando.

6. Arquitectura de la Solucion

El sistema se estructura bajo un esquema clasico de tres capas, patron arquitectonico ampliamente aceptado en el desarrollo de aplicaciones web por su claridad de separacion de responsabilidades y su facilidad de mantenimiento. Cada una de estas capas se ejecuta en el mismo proceso local durante el desarrollo, pero esta disenada de modo que podria separarse en procesos o servidores independientes sin modificar la logica de cada una de ellas.

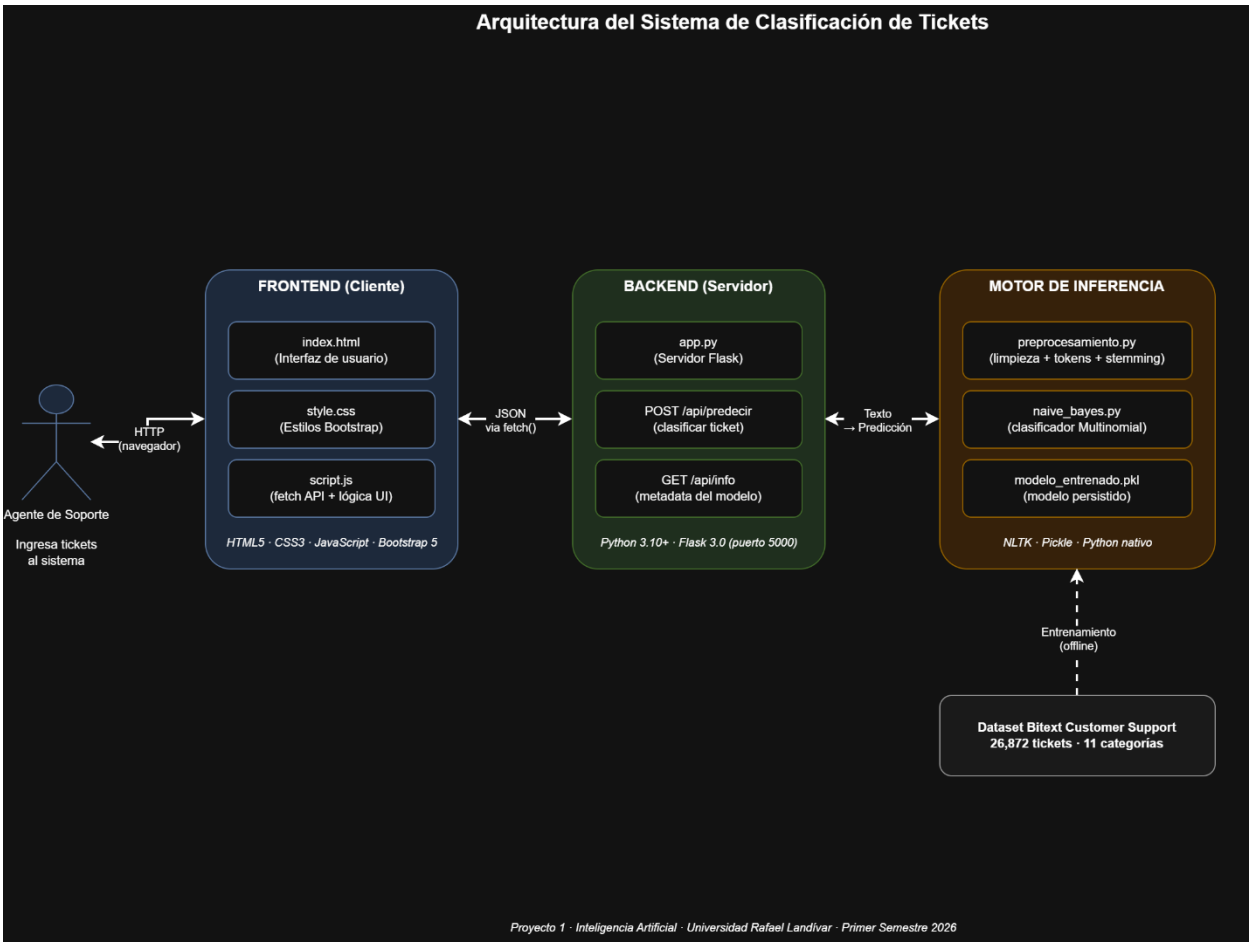

Figura 2. Arquitectura general del sistema de clasificacion de tickets.

6.1 Capa de Presentacion (Frontend)

Esta capa es la unica con la que el usuario interactua directamente. Esta compuesta por tres archivos principales: index.html, que define la estructura de la pagina y contiene el formulario de ingreso de tickets; style.css, que implementa los estilos visuales

personalizados sobre la base de Bootstrap 5; y script.js, que contiene toda la logica de interactividad, incluyendo la validacion del formulario, el envio de la peticion al backend mediante fetch() y la renderizacion dinamica de los resultados. El frontend se ejecuta completamente en el navegador del usuario, sin necesidad de plugins adicionales, y es compatible con cualquier navegador moderno (Chrome, Firefox, Edge, Safari).

6.2 Capa de Servicio (Backend)

El backend consiste en un unico archivo Python llamado app.py que utiliza el framework Flask para exponer dos endpoints REST. El primer endpoint, POST /api/predecir, recibe una peticion con el asunto y la descripcion del ticket, invoca el pipeline de preprocesamiento y clasificacion, y devuelve una respuesta JSON con la categoria predicha, la confianza y las probabilidades por clase. El segundo endpoint, GET /api/info, devuelve metadata del modelo (numero de instancias de entrenamiento, numero de clases, tamano del vocabulario y parametro alpha de Laplace) y se utiliza principalmente para fines de depuracion y para mostrar informacion en el pie de la pagina.

Una decision arquitectonica importante es que el modelo se carga una unica vez al iniciar el servidor y permanece en memoria durante toda la vida del proceso. Esto evita el costo de leer y deserializar el archivo pickle en cada peticion, lo cual incrementaria dramaticamente el tiempo de respuesta. La carga inicial toma aproximadamente dos segundos, pero a partir de ese momento cada prediccion tarda menos de diez milisegundos.

6.3 Capa de Inferencia (Motor)

El motor de inferencia es el corazon del proyecto y se compone de varios modulos de Python que trabajan en conjunto. El modulo preprocesamiento.py contiene la clase Preprocesador que encapsula todas las operaciones de limpieza textual. El modulo naive_bayes.py contiene la clase NaiveBayesMultinomial que implementa el entrenamiento y la inferencia del algoritmo. Los modulos kfolds.py y metricas.py implementan la evaluacion del modelo. Finalmente, el archivo modelo_entrenado.pkl almacena el modelo ya entrenado junto con el preprocesador y los metadatos, listo para ser cargado por el backend.

6.4 Flujo de Datos y Separacion de Preocupaciones

Una caracteristica importante de esta arquitectura es que las tres capas se comunican exclusivamente mediante interfaces bien definidas: el frontend habla con el backend a traves de peticiones HTTP con payload JSON, y el backend habla con el motor mediante llamadas a funciones Python. Esta separacion permite, por ejemplo, reemplazar el motor Naive Bayes por otro algoritmo de clasificacion (una red neuronal, un SVM, un clasificador de Random Forest) sin modificar ni una sola linea del frontend o del backend, siempre que la nueva implementacion respete la misma interfaz predecir_con_probabilidades(tokens). De forma analoga, el frontend podria reemplazarse por una aplicacion movil nativa o por un bot de chat sin tocar el motor de inferencia.

7. Preprocesamiento de Texto

El preprocesamiento es una etapa fundamental en cualquier sistema de procesamiento de lenguaje natural. Su proposito es transformar el texto crudo, con toda su variabilidad y ruido caracteristicos, en una representacion limpia, consistente y estructurada que el algoritmo de clasificacion pueda analizar de forma efectiva. Un preprocesamiento bien disenado puede marcar la diferencia entre un modelo que funciona y uno que fracasa, incluso cuando se utiliza el mismo algoritmo subyacente. En este proyecto se implemento un pipeline de preprocesamiento de cinco etapas, encapsulado en la clase Preprocesador del modulo preprocesamiento.py.

7.1 Etapa 1: Limpieza con Expresiones Regulares

La primera operacion consiste en eliminar o normalizar fragmentos del texto que no aportan informacion semantica. Se aplican en orden las siguientes expresiones regulares:

- **Placeholders del dataset:** el patron \{\{[^}]+\}\} captura textos como {{Order Number}} o {{Account Type}}, que son marcadores de datos que en un sistema real se rellenarian con valores concretos. Estos placeholders aparecen con frecuencia identica en todas las clases y actuarian como ruido si no se eliminaran.
- **URLs:** el patron http\S+|www\.\S+ captura direcciones web. Las URLs son casi siempre unicas por ticket y no contribuyen a identificar la categoria.
- **Direcciones de correo electronico:** el patron \S+@\S+ captura direcciones de correo. Al igual que las URLs, son identificadores unicos que no aportan al clasificador.
- **Numeros:** el patron \d+ elimina cualquier secuencia numerica. Los numeros en los tickets representan montos, identificadores de orden, fechas, cantidades; son extremadamente variables y rara vez repiten valores, lo que inflaria el vocabulario con tokens que aparecen una sola vez.
- **Caracteres especiales:** el patron [^a-zA-Z\s] elimina cualquier caracter que no sea una letra o un espacio. Esto incluye signos de puntuacion, simbolos, emoticonos y cualquier caracter no ASCII.

- **Espacios multiples:** el patron \s+ colapsa cualquier secuencia de espacios en blanco (espacios, tabulaciones, saltos de linea) en un unico espacio. Esto es necesario porque las operaciones anteriores dejan espacios huerfanos donde solian estar los elementos eliminados.

7.2 Etapa 2: Conversion a Minusculas

Todo el texto se convierte a minusculas con la funcion lower(). Sin esta normalizacion, el modelo tratariaa las palabras 'Order', 'order' y 'ORDER' como tres palabras distintas, lo cual inflaria el vocabulario y dividira artificialmente la evidencia estadistica. En el dominio de clasificacion de tickets, no existe diferencia semantica entre una palabra escrita con mayusculas y la misma palabra en minusculas, por lo que esta normalizacion es apropiada. Para otros dominios (como la identificacion de entidades nombradas) esta decision podria no ser adecuada, pero para nuestro caso si lo es.

7.3 Etapa 3: Tokenizacion

La tokenizacion consiste en separar el texto continuo en una lista de palabras individuales, denominadas tokens. Se utilizo la funcion word_tokenize de NLTK, que implementa un tokenizador basado en reglas mas sofisticado que un simple split por espacios. Este tokenizador maneja correctamente casos que un split ingenuo manejaria mal, como contracciones (don't se separa en do y n't), puntuacion pegada a palabras, y abreviaturas. Aunque en este pipeline muchos de esos casos ya fueron neutralizados por las expresiones regulares de limpieza, utilizar el tokenizador estandar garantiza robustez en cualquier eventualidad.

7.4 Etapa 4: Eliminacion de Stopwords

Las stopwords o palabras vacias son aquellas que aparecen con alta frecuencia en cualquier texto pero que aportan poca o ninguna informacion distintiva sobre la categoria del documento. Ejemplos en ingles son 'the', 'is', 'at', 'which', 'on', 'a', 'an', 'to'. Si se mantuvieran en el vocabulario, estas palabras dominarian las estadisticas de frecuencia sin aportar poder predictivo, ademas de hacer los calculos de verosimilitud innecesariamente costosos.

Se utilizo la lista de stopwords en ingles proporcionada por NLTK (stopwords.words('english')), que contiene aproximadamente 180 palabras. El filtrado se realiza mediante una comprension de lista que conserva unicamente los tokens que no estan en el conjunto de stopwords. Adicionalmente, se descartan los tokens de un solo caracter (como 's' resultante de contracciones del tipo it's que se tokenizan como it + 's'), ya que estos tokens residuales no aportan informacion util.

7.5 Etapa 5: Stemming

El stemming es un proceso de reduccion morfologica que busca agrupar diferentes variantes de una misma palabra a una raiz comun. Por ejemplo, las palabras 'running', 'runs' y 'ran' deberian ser tratadas como la misma unidad semantica (correr), a pesar de que son tres cadenas de texto distintas. Sin stemming, el vocabulario se multiplicaria con multiples variantes de la misma palabra, dividiendo la evidencia estadistica y reduciendo la capacidad generalizadora del modelo.

Se eligio el algoritmo de Porter (implementado en la clase PorterStemmer de NLTK), un algoritmo clasico del procesamiento de lenguaje natural desarrollado originalmente por Martin Porter en 1980. El algoritmo aplica una serie de reglas de sufijos para reducir una palabra a su raiz, por ejemplo eliminando '-ing', '-ed', '-ly' y otros sufijos comunes. Una caracteristica peculiar del stemming es que la raiz resultante no tiene que ser una palabra valida del idioma: 'subscription' se reduce a 'subscript' y 'changing' se reduce a 'chang'. Lo importante no es que la raiz sea legible para un humano, sino que sea consistente: todas las variantes morfologicas de 'subscription' (subscriptions, subscribed, subscribing) terminan en 'subscript', permitiendo al modelo agruparlas bajo un mismo token.

Se eligio stemming sobre lematizacion porque el stemming es mas rapido (no requiere consultar un diccionario) y suele ser suficiente para tareas de clasificacion donde solo importa la consistencia. La lematizacion, aunque mas precisa, requiere mas recursos computacionales y un diccionario de referencias.

7.6 Ejemplo Completo del Pipeline

A continuacion se muestra un ejemplo real de como un texto crudo es transformado a traves de las cinco etapas del pipeline. Se toma una solicitud tipica del dataset y se muestra el resultado despues de cada operacion:

```
Entrada: "I need to change the shipping address for order {{Order
   Number}}"  Despues de limpieza regex:  "i need to change the
   shipping address for order" Despues de minusculas:      (sin
      cambio, ya estaba en minusculas) Despues de tokenizar:
   ['i', 'need', 'to', 'change', 'the', 'shipping', 'address',
 'for', 'order'] Despues de quitar stopwords: ['need', 'change',
      'shipping', 'address', 'order'] Despues de stemming:
         ['need', 'chang', 'ship', 'address', 'order']
```

El resultado final es una lista de cinco tokens normalizados, lista para ser alimentada al clasificador Naive Bayes. Observese como el texto original de once palabras se redujo a cinco tokens significativos, eliminando ruido y conservando solo la informacion relevante para la clasificacion.

7.7 Decisiones de Diseno Relevantes

Durante el diseno del preprocesador se tomaron varias decisiones que conviene justificar explicitamente. Primero, todas las expresiones regulares se compilan una unica vez en el constructor de la clase, evitando recompilarlas en cada invocacion de la funcion procesar(). Esto es una optimizacion menor pero relevante dado que el preprocesador se aplica a mas de 26,000 documentos durante el entrenamiento.

Segundo, el orden de aplicacion de las expresiones regulares es critico: los placeholders con formato {{...}} deben eliminarse antes de eliminar los caracteres especiales (que incluyen las llaves), porque si se hiciera al reves se perderia la informacion de donde estaban los placeholders. Tercero, la conversion a minusculas se hace despues de las expresiones regulares de limpieza, no antes, para garantizar que los patrones de URLs y emails se detecten correctamente independientemente del caso.

Cuarto, el preprocesador se implementa como una clase (no como funciones sueltas) para poder ser serializado junto con el modelo en el archivo pickle. Esto es fundamental: la aplicacion web debe aplicar exactamente las mismas transformaciones que se

aplicaron durante el entrenamiento, de lo contrario los tokens no coincidirian con el vocabulario aprendido. Al serializar el preprocesador junto con el modelo, garantizamos esta consistencia.

8. Algoritmo Naive Bayes - Explicacion Matematica

Naive Bayes es una familia de clasificadores probabilisticos basados en la aplicacion del teorema de Bayes bajo una suposicion fuerte (e irrealista) de independencia entre las caracteristicas. A pesar de esta suposicion, Naive Bayes ha demostrado ser sorprendentemente eficaz en diversas tareas de clasificacion, particularmente en clasificacion de texto, donde sus resultados compiten con algoritmos mucho mas complejos. Esta seccion presenta la formulacion matematica completa del algoritmo tal como fue implementado en el proyecto.

8.1 El Teorema de Bayes

El teorema de Bayes, formulado por Thomas Bayes en el siglo XVIII, es la base matematica de todo clasificador bayesiano. Para el problema de clasificacion de texto, el teorema se expresa como:

$$P(C \mid X) = [\, P(X \mid C) * P(C) \,] \,/\, P(X)$$

Donde C representa una clase (por ejemplo, ORDER, REFUND, etc.) y X representa un documento (un ticket en nuestro caso). La interpretacion de cada termino es la siguiente:

- **P(C | X):** probabilidad posterior. Es la probabilidad de que la clase correcta sea C dado que hemos observado el documento X. Este es el valor que queremos calcular.
- **P(X | C):** verosimilitud. Es la probabilidad de generar el documento X si la clase realmente fuera C. Se estima a partir de los datos de entrenamiento.
- **P(C):** probabilidad a priori de la clase. Representa la frecuencia relativa de cada clase en el conjunto de entrenamiento.
- **P(X):** probabilidad marginal del documento. Actua como un factor de normalizacion. Dado que es la misma para todas las clases (no depende de C), se puede ignorar cuando el objetivo es elegir la clase de mayor probabilidad.

Por lo tanto, para clasificacion solo necesitamos calcular el numerador y seleccionar la clase que maximice el producto P(X|C) * P(C). Esto se conoce como la regla de decision de maxima a posteriori (MAP):

$$C^* = argmax_C \,[\, P(X \mid C) * P(C) \,]$$

8.2 La Suposicion de Independencia (Por que 'Naive')

Calcular P(X|C) directamente es impracticable: un documento X consiste en una secuencia de palabras w1, w2, ..., wn, y la probabilidad de observar esa secuencia exacta en una clase dada requeriria un modelo conjunto extremadamente complejo que considerara todas las dependencias entre palabras. Aqui es donde entra la suposicion 'naive': se asume que todas las palabras son condicionalmente independientes dado la clase. En otras palabras, saber que el documento pertenece a la clase C, la ocurrencia de una palabra no afecta la probabilidad de ocurrencia de las demas.

Matematicamente, esta suposicion se expresa como:

$$P(X \mid C) = P(w1 \mid C) * P(w2 \mid C) * \ldots * P(wn \mid C)$$

Esta es una suposicion evidentemente falsa en la practica: las palabras 'credit' y 'card' aparecen juntas con mucha mayor frecuencia de la que aparecerian si fueran independientes. Sin embargo, la suposicion simplifica enormemente el calculo al convertir un problema de estimacion conjunta intratable en un conjunto de estimaciones univariadas facilmente calculables. Lo mas sorprendente es que, a pesar de la falsedad de la suposicion, Naive Bayes funciona bien en la practica. Esto se debe a que, para decisiones de clasificacion, no importan tanto los valores exactos de las probabilidades como las relaciones de orden entre ellas: aunque las probabilidades calculadas pueden estar sesgadas, la clase ganadora suele ser la misma que con un modelo mas sofisticado.

8.3 Naive Bayes Multinomial

Existen varias variantes de Naive Bayes dependiendo de como se modele la probabilidad de las caracteristicas dado la clase. Las dos variantes mas comunes para clasificacion de texto son la Bernoulli (modela la presencia o ausencia de cada palabra como una variable binaria) y la Multinomial (modela la frecuencia de aparicion de cada palabra como un conteo entero). En este proyecto se utilizo la variante Multinomial porque generalmente rinde mejor en clasificacion de texto, ya que aprovecha la informacion de repeticion de palabras: un documento que menciona 'refund' cinco veces es mas probablemente un ticket de reembolso que uno que lo menciona solo una vez.

La probabilidad de una palabra dado una clase en el modelo multinomial se estima como:

$$P(w \mid C) = count(w, C) \,/\, SUMA_\{w' \text{ in } V\}\ count(w', C)$$

Donde count(w, C) es el numero de veces que la palabra w aparece en todos los documentos de la clase C en el conjunto de entrenamiento, y el denominador es el total de palabras (con repeticion) en todos los documentos de la clase C. La suma se extiende sobre todas las palabras del vocabulario V.

8.4 Laplace Smoothing (Suavizado Aditivo)

La formula anterior presenta un problema serio: si una palabra del documento de prueba nunca aparecio en los documentos de una clase especifica durante el entrenamiento, entonces P(w|C) = 0, y como las verosimilitudes se multiplican, toda la probabilidad posterior colapsa a cero. Esto es particularmente grave porque una sola palabra desconocida puede descartar por completo una clase que, por el resto del documento, seria la mas probable.

La solucion estandar a este problema es el suavizado de Laplace (tambien conocido como add-one smoothing o suavizado aditivo), que consiste en anadir una cantidad pequena (tipicamente 1) al numerador de cada palabra, y compensar en el denominador para mantener la propiedad de que las probabilidades sumen uno. La formula suavizada queda:

$$P(w \mid C) = [\, count(w, C) + alpha \,] \,/\, [\, SUMA_\{w' \text{ in } V\}\ count(w', C) + alpha * |V| \,]$$

Donde alpha es el parametro de suavizado (en nuestro caso alpha = 1, que es el clasico add-one smoothing) y |V| es el tamano del vocabulario. El efecto de este suavizado es que ninguna palabra del vocabulario tendra probabilidad cero en ninguna clase, por muy rara que sea; en el peor caso tendra probabilidad alpha / (N + alpha*|V|) donde N es el total de palabras de la clase. Esto estabiliza el clasificador y lo hace robusto ante palabras poco frecuentes.

El parametro alpha controla la fuerza del suavizado: valores mayores a 1 suavizan mas agresivamente (dando mas peso al uniforme y menos a los datos observados), mientras

que valores menores a 1 suavizan menos (confiando mas en los datos). El valor alpha=1 ofrece un balance razonable para datasets de tamano mediano como el nuestro.

8.5 Suma de Logaritmos (Evitando Underflow Numerico)

Al multiplicar muchas probabilidades pequenas (cada P(w|C) es tipicamente del orden de 10^-4 o menor), el producto final se vuelve extraordinariamente pequeno: para un documento con 50 palabras, el producto podria ser del orden de 10^-200, un valor que supera la precision de los numeros en punto flotante y es redondeado silenciosamente a cero por el hardware. Este fenomeno se llama underflow numerico y, si no se corrige, hace que el clasificador produzca probabilidades cero espurias para todas las clases.

La solucion estandar es trabajar con logaritmos de las probabilidades. Gracias a las propiedades de los logaritmos, el producto de probabilidades se convierte en una suma de logaritmos:

$$log\ P(X\ |\ C) = SUMA_i\ log\ P(wi\ |\ C)$$

Y la regla de decision se transforma en:

$$C^* = argmax_C\ [\ log\ P(C) + SUMA_i\ log\ P(wi\ |\ C)\ ]$$

Los logaritmos de probabilidades son numeros negativos (ya que las probabilidades estan entre 0 y 1), pero ya no sufren de underflow: un numero como -500 es perfectamente representable en punto flotante de doble precision, mientras que exp(-500) es indistinguible de cero. Ademas, como la funcion logaritmo es monotona creciente, la clase que maximiza log P(C|X) tambien maximiza P(C|X), por lo que la decision final es correcta.

En nuestra implementacion, los logaritmos se calculan una unica vez durante el entrenamiento y se almacenan como valores precalculados. Durante la inferencia, solo se realizan sumas (no productos y no logaritmos), lo que tambien es mas eficiente computacionalmente.

8.6 Normalizacion de Probabilidades (Log-Sum-Exp Trick)

Aunque para la clasificacion basta con comparar los log-probabilidades y tomar el argumento maximo, en el proyecto tambien se calculan las probabilidades normalizadas por clase (valores entre 0 y 1 que suman 1) para mostrarlas al usuario en la interfaz web como barras de confianza. El paso de los log-probabilidades a probabilidades normalizadas no es trivial: si simplemente calculamos exp(log_p) para cada clase, podemos volver a caer en underflow.

La solucion es el log-sum-exp trick, una tecnica numerica estandar. Dado el conjunto de log-probabilidades {log_p_1, log_p_2, ..., log_p_k}, primero encontramos el maximo L = max(log_p_i) y luego calculamos las probabilidades normalizadas como:

$$p_i = exp(log_p_i - L) / SUMA_j\ exp(log_p_j - L)$$

Al restar el maximo antes de aplicar exp(), garantizamos que todos los argumentos de exp son menores o iguales a cero (y por tanto los valores estan entre 0 y 1), evitando tanto underflow como overflow. El factor exp(L) se cancela entre numerador y denominador, por lo que el resultado es matematicamente identico a la formula sin el truco, pero numericamente estable.

8.7 Algoritmo Completo de Inferencia

Juntando todas las piezas, el algoritmo completo de inferencia para un documento nuevo X se resume en los siguientes pasos:

- Preprocesar el documento X aplicando el pipeline descrito en la seccion 7, obteniendo una lista de tokens.
- Para cada clase C en el conjunto de clases, calcular log P(C|X) como log P(C) + SUMA de log P(wi|C) para cada token wi del documento. Los tokens que no estan en el vocabulario se ignoran (no contribuyen a la suma).
- Seleccionar la clase C* que maximiza log P(C|X). Esta es la prediccion final.
- Opcionalmente, convertir los log-probabilidades a probabilidades normalizadas usando el log-sum-exp trick para mostrarlas al usuario.

El costo computacional de cada prediccion es O(n * k), donde n es el numero de tokens del documento y k es el numero de clases. Para nuestros parametros (n aprox 10, k =

11), esto se traduce en aproximadamente 110 operaciones de suma por prediccion, lo cual permite clasificar miles de documentos por segundo en hardware estandar.

9. Metricas de Evaluacion

Evaluar el rendimiento de un clasificador no es tan simple como contar aciertos. En problemas multiclase con clases desbalanceadas, una unica metrica puede resultar enganosa. Por ejemplo, en un dataset donde el 90% de los ejemplos pertenecen a una clase, un clasificador que siempre prediga esa clase alcanzaria 90% de accuracy sin haber aprendido nada util. Por ello, en este proyecto se emplean multiples metricas que ofrecen perspectivas complementarias del rendimiento del modelo. Esta seccion define formalmente cada metrica y explica su interpretacion.

9.1 Matriz de Confusion

La matriz de confusion es la herramienta fundamental para evaluar clasificadores multiclase. Se trata de una tabla bidimensional donde cada fila representa una clase real y cada columna representa una clase predicha por el modelo. La celda en la fila i, columna j contiene la cantidad de ejemplos cuya clase real es i pero que el modelo clasifico como j. La diagonal principal contiene los aciertos (real i clasificado como i), y todas las celdas fuera de la diagonal representan errores de clasificacion.

La matriz de confusion es especialmente util porque no solo informa cuantos errores comete el modelo, sino que tipo de errores comete. Dos clasificadores pueden tener la misma accuracy pero cometer errores completamente distintos: uno puede confundir sistematicamente SHIPPING con DELIVERY (un error semanticamente comprensible), mientras que otro puede confundir aleatoriamente cualquier clase con cualquier otra (un error inaceptable). La matriz de confusion revela este tipo de patrones.

9.2 Precision

La precision para una clase C se define como la fraccion de ejemplos predichos como C que realmente son C:

$$Precision(C) = TP(C) / [\, TP(C) + FP(C) \,]$$

Donde TP(C) son los verdaderos positivos (ejemplos que son realmente C y fueron clasificados correctamente como C) y FP(C) son los falsos positivos (ejemplos que no

son C pero el modelo los clasifico como C). Una precision alta significa que, cuando el modelo predice clase C, rara vez se equivoca. Es una metrica importante cuando el costo de un falso positivo es alto: por ejemplo, si un ticket incorrectamente clasificado como URGENTE desvia recursos del equipo de soporte de urgencias reales, preferimos alta precision en esa clase.

9.3 Recall (Sensibilidad)

El recall para una clase C se define como la fraccion de ejemplos que realmente son C que fueron correctamente identificados por el modelo:

$$Recall(C) = TP(C) / [\, TP(C) + FN(C) \,]$$

Donde FN(C) son los falsos negativos (ejemplos que son realmente C pero el modelo los clasifico como otra cosa). Un recall alto significa que el modelo detecta la mayor parte de los casos verdaderos de clase C. Es una metrica importante cuando el costo de perder un caso positivo es alto: por ejemplo, si un ticket de REEMBOLSO clasificado erroneamente como ORDER hace que el cliente nunca reciba su dinero de vuelta, preferimos alto recall en la clase REFUND.

9.4 F1-Score

Precision y recall suelen estar en tension: es posible obtener recall perfecto (100%) a costa de la precision (por ejemplo, prediciendo siempre clase C se garantiza no perder ningun caso positivo, pero la precision sera baja). Para tener una metrica unificada que balancee ambas dimensiones, se utiliza el F1-score, que es la media armonica entre precision y recall:

$$F1(C) = 2 * Precision(C) * Recall(C) / [\, Precision(C) + Recall(C) \,]$$

La media armonica penaliza mas severamente los valores bajos que la media aritmetica: si precision es 1.0 y recall es 0.1, la media aritmetica seria 0.55 (que sonaria aceptable), pero el F1 seria 0.18 (reflejando correctamente que el clasificador es malo). F1 alcanza su valor maximo (1.0) cuando precision y recall son ambos 1.0, y es cero si cualquiera de los dos es cero. Por esta razon F1 es probablemente la metrica mas importante por clase para evaluar clasificadores multiclase.

9.5 Accuracy Global

La accuracy global es simplemente la fraccion de predicciones correctas sobre el total:

$$Accuracy = Aciertos / Total$$

Aunque es la metrica mas intuitiva, tiene la limitacion conocida de poder ser enganosa en datasets desbalanceados. En este proyecto la reportamos porque ofrece una vision panoramica del rendimiento y porque el desbalance de nuestro dataset es moderado (la clase mayoritaria tiene solo 6 veces mas ejemplos que la minoritaria, no un factor de 10 o 100 como en problemas extremadamente desbalanceados).

9.6 Macro F1

El Macro F1 es el promedio simple (sin ponderar) del F1 de cada clase:

$$Macro\ F1 = (F1(C1) + F1(C2) + ... + F1(Ck)) / k$$

Esta metrica trata a todas las clases como igualmente importantes, independientemente de cuantos ejemplos tengan. Es por tanto particularmente adecuada para datasets desbalanceados: un clasificador que funcione bien en la clase mayoritaria pero mal en las minoritarias tendria alta accuracy pero bajo Macro F1, revelando el problema. En este sentido Macro F1 complementa a accuracy y juntas ofrecen una vision mas completa.

Una variante alternativa es el Weighted F1, que pondera cada F1 por la cantidad de ejemplos de esa clase. En este proyecto preferimos Macro F1 porque trata todas las clases con la misma importancia, lo cual es adecuado en un sistema de enrutamiento de tickets donde cada categoria representa un departamento real (no quieres que el sistema ignore a los departamentos con menos volumen).

10. K-Folds Cross Validation

Cuando se evalua un modelo de aprendizaje supervisado, la practica comun es separar el dataset en dos conjuntos disjuntos: uno para entrenamiento y uno para prueba. El modelo se ajusta con el conjunto de entrenamiento y se evalua con el conjunto de prueba, obteniendo metricas que reflejan su capacidad de generalizacion a datos no vistos. Sin embargo, esta tecnica (conocida como hold-out validation) tiene una debilidad importante: el resultado depende fuertemente de que ejemplos especificos fueron asignados a cada conjunto. Un particionado desafortunado puede hacer que el conjunto de prueba sea accidentalmente mas facil o mas dificil que el promedio, distorsionando las metricas.

10.1 Motivacion y Funcionamiento

La validacion cruzada K-Folds resuelve este problema repitiendo el proceso de evaluacion K veces con distintos particionados. El dataset se divide en K particiones (folds) de tamano aproximadamente igual. Luego se realizan K experimentos: en el experimento i, el fold i se utiliza como conjunto de prueba y los K-1 folds restantes como conjunto de entrenamiento. Cada ejemplo del dataset es utilizado exactamente una vez como caso de prueba y K-1 veces como caso de entrenamiento. Las metricas finales se obtienen promediando las metricas de los K experimentos.

Este procedimiento ofrece varias ventajas. En primer lugar, reduce la varianza de las estimaciones: en lugar de depender de un unico particionado aleatorio, se promedian K estimaciones, lo que atenua la influencia de ejemplos atipicos que pudieran caer en el conjunto de prueba. En segundo lugar, aprovecha mejor los datos disponibles: cada ejemplo es utilizado tanto para entrenar como para probar (aunque no simultaneamente). En tercer lugar, permite calcular la varianza entre folds, lo cual es un indicador util de la estabilidad del modelo: si los K experimentos producen metricas muy similares, el modelo es robusto; si producen metricas muy distintas, el modelo es sensible a los datos de entrenamiento especificos.

10.2 Eleccion de K = 5

El valor de K elegido para este proyecto fue 5, que es el minimo requerido por el enunciado y tambien el valor mas comun en la literatura. La eleccion de K involucra un balance: valores mas grandes (como K=10) producen estimaciones mas robustas (mas experimentos = menor varianza del promedio) pero son mas costosos computacionalmente (el modelo se entrena K veces). Valores mas pequenos (como K=3) son mas rapidos pero menos confiables. K=5 ofrece un balance razonable: cada iteracion entrena con el 80% de los datos y prueba con el 20%, proporciones que se consideran adecuadas para la mayoria de los problemas.

10.3 Implementacion Manual

El proyecto requiere explicitamente que la implementacion de K-Folds sea manual, sin utilizar la funcion KFold de sklearn.model_selection. La implementacion propia se encuentra en el modulo kfolds.py y sigue los siguientes pasos:

- Los indices del dataset (0 a N-1) se mezclan aleatoriamente usando un generador con semilla fija (seed=42) para garantizar reproducibilidad. Sin este mezclado, los folds podrian quedar desbalanceados si el dataset estuviera ordenado por clase.
- Los indices mezclados se dividen en K = 5 folds de tamano aproximadamente igual. El ultimo fold se lleva los ejemplos sobrantes cuando N no es exactamente divisible por K.
- Se ejecutan K iteraciones. En cada iteracion, uno de los folds se reserva como conjunto de prueba y el resto se concatena como conjunto de entrenamiento.
- Para cada iteracion se entrena un modelo Naive Bayes completamente nuevo (no se reutilizan modelos entre iteraciones para evitar data leakage) y se evaluan sus predicciones sobre el conjunto de prueba.
- Al finalizar las K iteraciones, se promedian las metricas obtenidas y se calcula la varianza entre folds como indicador de estabilidad.

10.4 Prevencion de Data Leakage

Un error comun en la validacion cruzada es el llamado data leakage, que ocurre cuando informacion del conjunto de prueba se filtra al conjunto de entrenamiento, inflando artificialmente las metricas. En nuestro caso, tomamos la precaucion de que el vocabulario y los conteos de frecuencia se reconstruyen desde cero en cada fold,

utilizando unicamente los datos del conjunto de entrenamiento correspondiente. Si construyeramos el vocabulario globalmente antes del K-Folds, estariamos usando informacion de los ejemplos de prueba (aunque fuera indirectamente) para entrenar al modelo.

Por otro lado, el preprocesamiento del texto (limpieza regex, tokenizacion, stemming) si se aplica una unica vez antes del K-Folds, por una razon tecnica: el preprocesamiento es deterministico y no depende de los datos de entrenamiento, solo transforma cada texto de forma aislada. Aplicarlo globalmente ahorra tiempo de computo significativo sin introducir leakage. La regla general es: cualquier paso que aprenda parametros a partir de los datos (como construir el vocabulario) debe ejecutarse dentro del bucle de K-Folds, mientras que los pasos deterministicos que no aprenden nada de los datos pueden ejecutarse fuera.

11. Diagrama de Casos de Uso

El diagrama de casos de uso UML representa las interacciones entre los actores externos y el sistema de clasificacion de tickets. Su funcion es documentar todas las funcionalidades que el sistema ofrece desde la perspectiva de los usuarios, sin entrar aun en detalles de implementacion. Para este proyecto se identificaron dos actores principales: el Agente de Soporte, que es el usuario final que ingresa tickets a traves de la interfaz web, y el Administrador del Sistema, que es responsable del mantenimiento del modelo (entrenamiento, consulta de metadata).

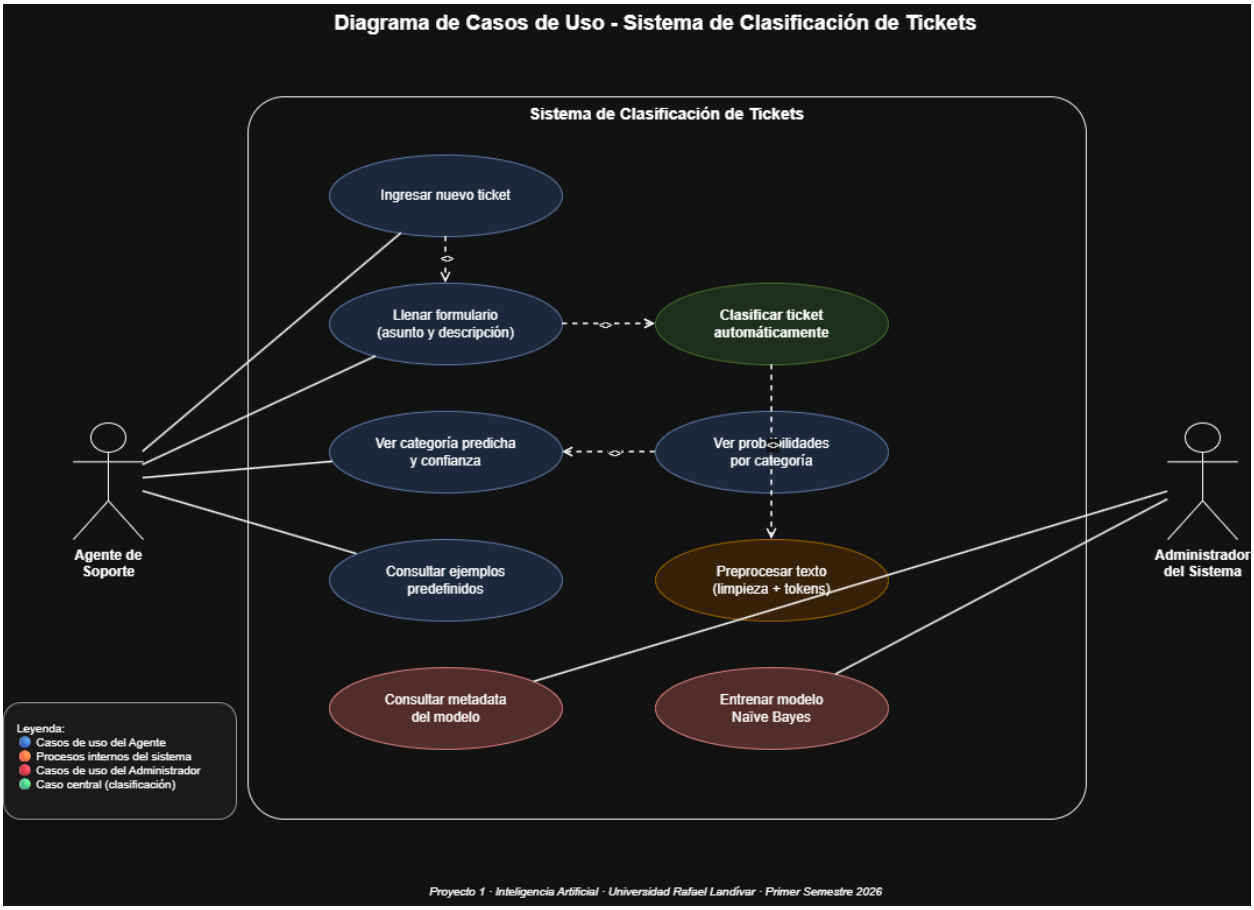

Figura 3. Diagrama de casos de uso del sistema.

El caso de uso central es 'Clasificar ticket automaticamente', que se activa como consecuencia inmediata del ingreso de un nuevo ticket (relacion de inclusion). A su vez, este caso depende de otro caso de uso de menor nivel ('Preprocesar texto') que se ejecuta como parte integral del proceso de clasificacion. El caso 'Ver probabilidades por

categoria' extiende el caso 'Ver categoria predicha': es una funcionalidad opcional que amplia el resultado basico con informacion adicional de confianza.

El administrador tiene acceso a dos casos de uso de uso infrecuente pero importante: 'Entrenar modelo Naive Bayes' (que se ejecuta una unica vez durante el despliegue inicial del sistema, o cuando se desea actualizar el modelo con nuevos datos) y 'Consultar metadata del modelo' (accesible a traves del endpoint GET /api/info). Esta separacion de roles refleja una practica comun en sistemas de produccion: el usuario final no necesita (ni debe) tener acceso a operaciones de administracion.

12. Diagrama de Flujo General

El diagrama de flujo muestra la secuencia completa de pasos que se ejecutan desde que el usuario abre la aplicacion web hasta que visualiza el resultado de la clasificacion. Este diagrama es util para comprender el comportamiento dinamico del sistema y para identificar los puntos de decision donde el flujo puede bifurcarse.

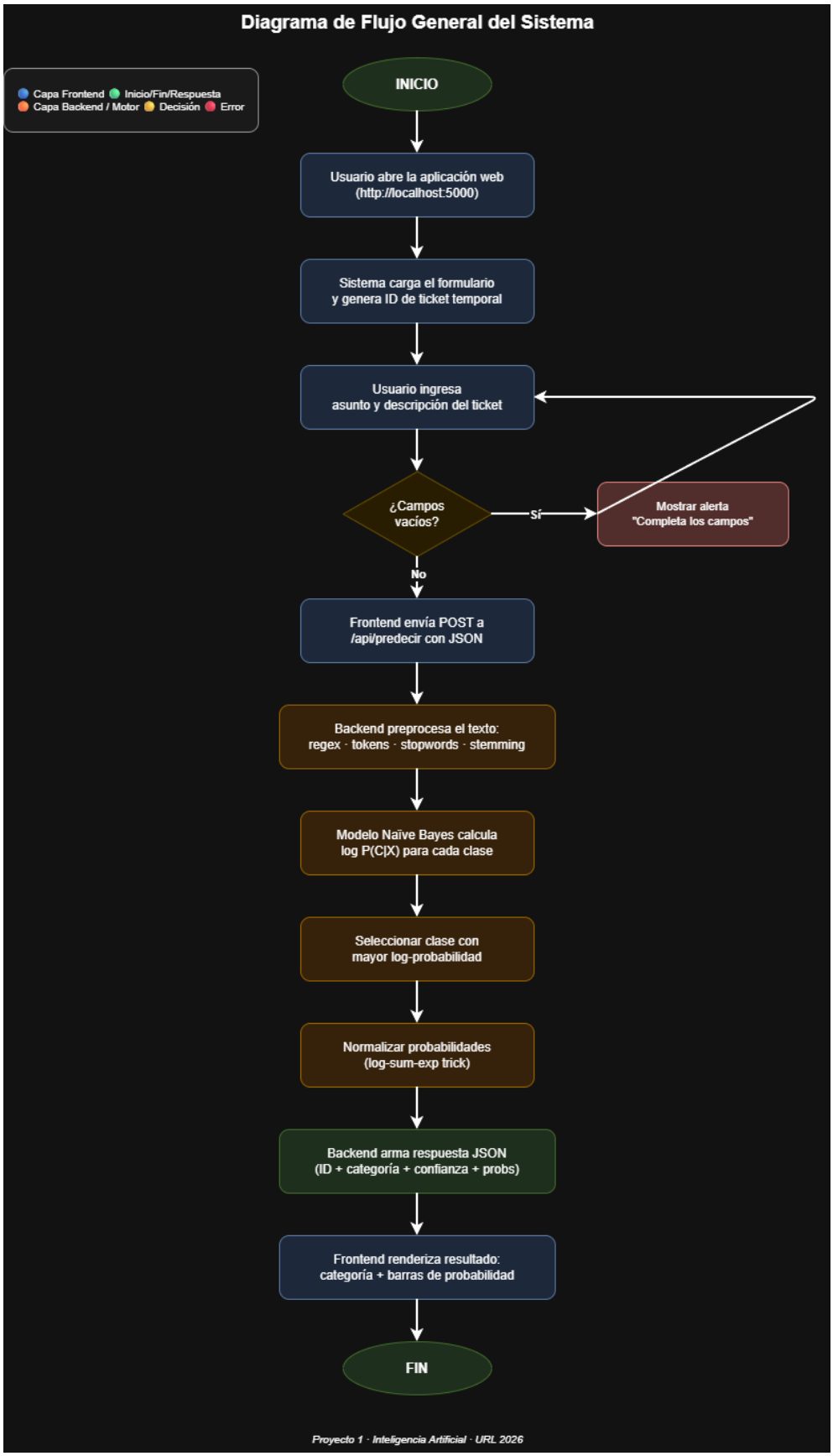
Diagrama de Flujo General del Sistema
Capa Frontend
Inicio/Fin/Respuesta
Capa Backend / Motor
Decisión
Error
INICIO
Usuario abre la aplicación web (http://localhost:5000)
Sistema carga el formulario y genera ID de ticket temporal
Usuario ingresa asunto y descripción del ticket
¿Campos vacíos?
Sí
Mostrar alerta "Completa los campos"
No
Frontend envía POST a /api/predecir con JSON
Backend preprocesa el texto: regex · tokens · stopwords · stemming
Modelo Naïve Bayes calcula log P(C|X) para cada clase
Seleccionar clase con mayor log-probabilidad
Normalizar probabilidades (log-sum-exp trick)
Backend arma respuesta JSON (ID + categoría + confianza + probs)
Frontend renderiza resultado: categoría + barras de probabilidad
FIN
Proyecto 1 · Inteligencia Artificial · URL 2026

Figura 4. Flujo general del sistema de clasificacion.

El flujo comienza cuando el usuario abre la URL del servidor Flask en su navegador. La pagina carga el formulario de tickets y se genera una etiqueta de ticket provisional. El usuario completa los campos de asunto y descripcion. Al presionar el boton Enviar, el sistema realiza una primera validacion en el lado del cliente: si ambos campos estan vacios, se muestra una alerta y se regresa al formulario. Si la validacion pasa, el frontend envia una peticion POST al endpoint /api/predecir con un payload JSON.

En el lado del servidor, Flask recibe la peticion, invoca al preprocesador sobre el texto combinado (asunto + descripcion), obtiene la lista de tokens, y la pasa al clasificador Naive Bayes. El clasificador calcula log P(C|X) para cada clase, elige el argmax como prediccion y normaliza las probabilidades mediante log-sum-exp. El backend genera un ID de ticket unico (formato TKT-AAAAMMDD-XXXXX) y construye la respuesta JSON con todos los datos relevantes. Finalmente, el frontend recibe la respuesta y renderiza visualmente el resultado: muestra la categoria predicha en un badge, la confianza en porcentaje, y las probabilidades por clase como barras de progreso ordenadas.

13. Diagrama de Componentes

El diagrama de componentes UML ofrece una vista estatica de la descomposicion del sistema en modulos de software y las dependencias entre ellos. A diferencia del diagrama de arquitectura (que muestra las capas de alto nivel) y del diagrama de secuencias (que muestra el comportamiento dinamico), este diagrama se enfoca en la estructura fisica del codigo: que modulos existen, en que paquete logico se agrupan y quien depende de quien.

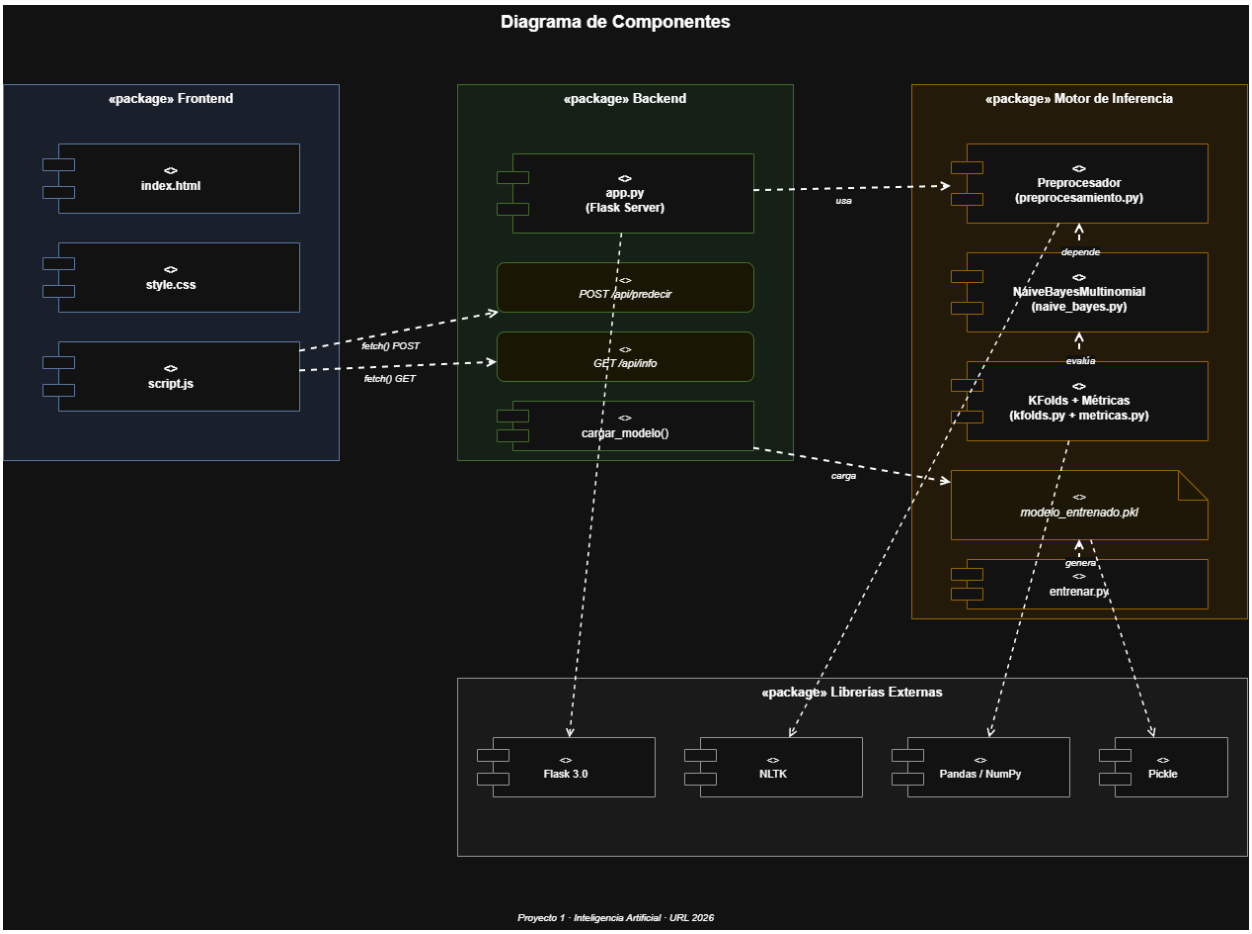

Figura 5. Diagrama de componentes del sistema.

El paquete Frontend agrupa los tres componentes que se ejecutan en el navegador del cliente. Estos no tienen dependencias entre si a nivel de codigo (aunque index.html incluye a los otros dos en tiempo de ejecucion mediante etiquetas link y script). El paquete Backend contiene el componente principal app.py, que expone dos interfaces (marcadas con el estereotipo UML <<interface>>): los endpoints POST /api/predecir y

GET /api/info. El componente cargar_modelo se ejecuta una unica vez al iniciar la aplicacion y carga el archivo pickle en memoria.

El paquete Motor de Inferencia agrupa los componentes relacionados con el algoritmo en si. Preprocesador y NaiveBayesMultinomial son los dos componentes centrales, donde el segundo depende del primero. El componente KFolds+Metricas solo se utiliza durante la fase de evaluacion, no durante la inferencia. El artefacto modelo_entrenado.pkl (representado con el estereotipo UML <<artifact>>) es el resultado de invocar al componente entrenar.py y es lo que el backend carga al iniciar el servidor.

Finalmente, el paquete Librerias Externas agrupa las dependencias de terceros: Flask (usado por el backend), NLTK (usado por el preprocesador), Pandas/NumPy (usados por el motor durante entrenamiento y evaluacion) y Pickle (usado para serializacion). Todas son dependencias estables y bien mantenidas.

14. Diagrama de Secuencias

El diagrama de secuencias UML representa la comunicacion entre las distintas partes del sistema a lo largo del tiempo durante un unico caso de uso: la clasificacion de un ticket. A diferencia de los diagramas anteriores (que son estaticos), este diagrama se enfoca en el orden temporal de los mensajes intercambiados entre objetos o componentes.

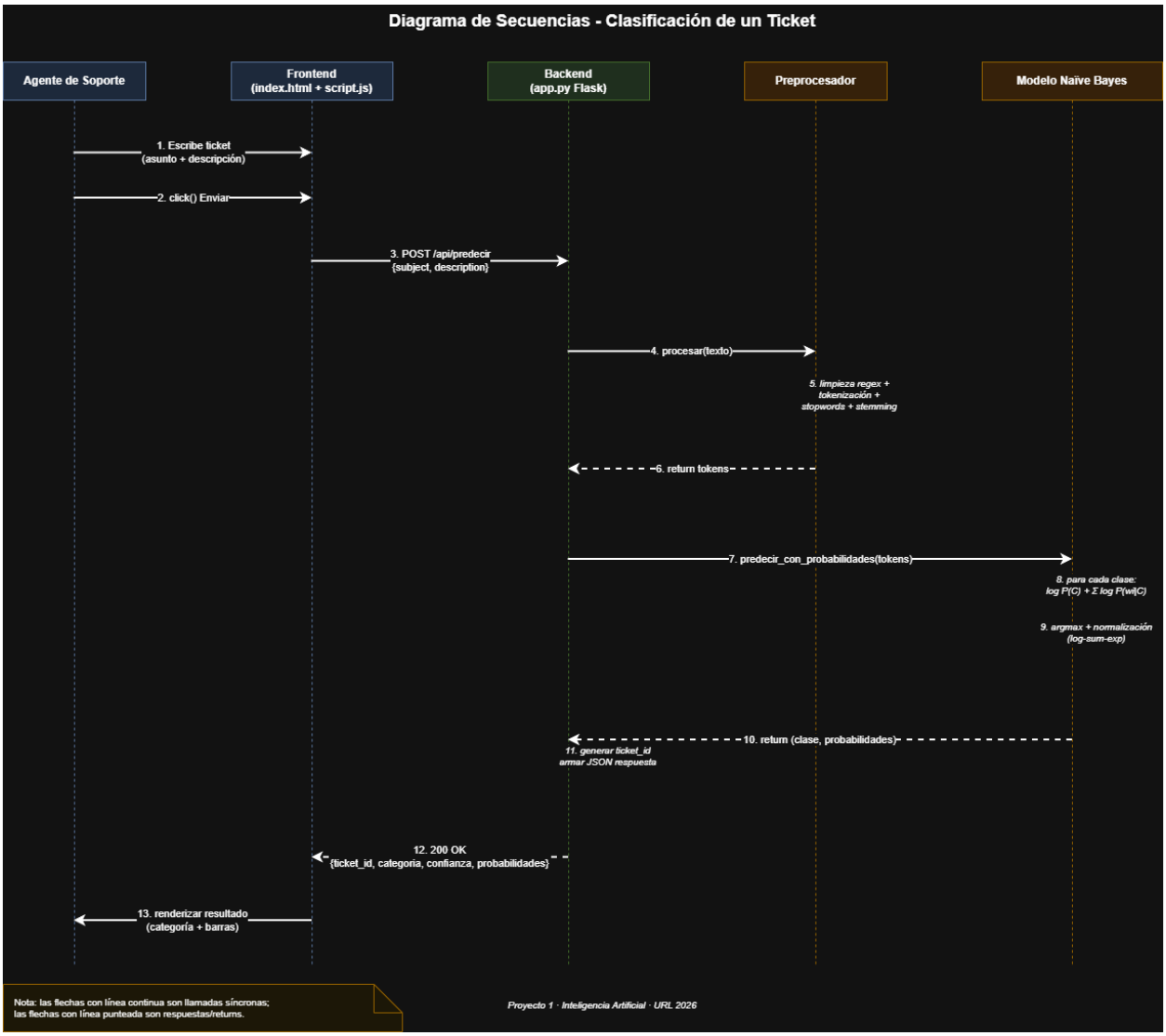

Figura 6. Diagrama de secuencias - clasificacion de un ticket.

El diagrama representa cinco lineas de vida verticales (lifelines) correspondientes a los actores y componentes que participan en la interaccion: el Agente de Soporte (usuario humano), el Frontend (HTML + JS), el Backend (Flask), el Preprocesador y el Modelo

Naive Bayes. El tiempo fluye de arriba hacia abajo, y las flechas horizontales representan los mensajes intercambiados.

La interaccion completa consiste en 13 mensajes. Los primeros dos son interacciones del usuario con el frontend (escribir el ticket, hacer click en Enviar). El mensaje 3 es la peticion HTTP del frontend al backend. Los mensajes 4 a 6 corresponden al preprocesamiento del texto. Los mensajes 7 a 10 son la invocacion al clasificador Naive Bayes y el retorno de la prediccion con las probabilidades. Los mensajes 11 a 13 cierran el ciclo: el backend construye la respuesta JSON, la envia al frontend y el frontend la renderiza visualmente para el usuario.

Las flechas con linea solida representan llamadas sincronas (el emisor espera la respuesta antes de continuar), mientras que las flechas con linea punteada representan los retornos o respuestas. Esta convencion es estandar en UML y permite distinguir claramente los mensajes de ida y vuelta.

15. Resultados Obtenidos

Esta seccion presenta los resultados cuantitativos de la evaluacion del modelo Naive Bayes sobre el dataset Bitext Customer Support, obtenidos mediante la tecnica de validacion cruzada K-Folds con K=5 descrita previamente. Los resultados se organizan en cuatro apartados: metricas por fold, metricas por clase, matriz de confusion y analisis de varianza entre folds.

15.1 Resultados por Fold

La siguiente tabla presenta la accuracy global y el macro F1-score obtenidos en cada uno de los cinco folds ejecutados. Cada fold utilizo aproximadamente 21,498 ejemplos para entrenamiento y 5,374 para prueba (con el ultimo fold teniendo ligeramente mas por redondeo).

Fold	Tamano Train	Tamano Test	Accuracy / Macro F1
1	21,498	5,374	0.9913 / 0.9911
2	21,498	5,374	0.9927 / 0.9924
3	21,498	5,374	0.9920 / 0.9917
4	21,498	5,374	0.9905 / 0.9902
5	21,496	5,376	0.9894 / 0.9891
Promedio	-	-	**0.9912 / 0.9909**

Tabla 1. *Metricas obtenidas en cada fold durante la validacion cruzada.*

La accuracy promedio del modelo es 0.9912 (99.12%), con una desviacion estandar de apenas 0.0012 entre folds. El macro F1 promedio es 0.9909, tambien con una desviacion estandar minima de 0.0012. Estos resultados son notablemente altos y consistentes. En las siguientes subsecciones profundizamos en las metricas por clase y en la interpretacion de estos numeros.

15.2 Metricas por Clase

A continuacion se presentan los resultados desglosados por cada una de las once categorias. Las metricas mostradas son los promedios a traves de los cinco folds de validacion cruzada.

Clase	Precision	Recall	F1-Score
ACCOUNT	0.9968	0.9988	0.9978
CANCEL	1.0000	0.9958	0.9979
CONTACT	0.9995	0.9975	0.9985
DELIVERY	0.9819	0.9274	0.9539
FEEDBACK	0.9955	0.9990	0.9972
INVOICE	0.9995	0.9915	0.9955
ORDER	0.9765	0.9925	0.9844
PAYMENT	0.9975	0.9966	0.9971
REFUND	0.9973	0.9960	0.9966
SHIPPING	0.9680	0.9974	0.9825
SUBSCRIPTION	1.0000	0.9971	0.9986

Tabla 2. *Metricas de precision, recall y F1 por clase (promedio de 5 folds).*

Observando la tabla, se identifican varios patrones interesantes. Las clases CANCEL y SUBSCRIPTION alcanzan una precision perfecta de 1.0000, lo que significa que el modelo nunca predice falsos positivos para estas clases. Las clases con F1-score mas alto son SUBSCRIPTION (0.9986), CONTACT (0.9985) y CANCEL (0.9979), todas por encima de 0.9975. Por otro lado, las clases con F1-score mas bajo son DELIVERY (0.9539), SHIPPING (0.9825) y ORDER (0.9844). Notablemente, DELIVERY tiene el recall mas bajo (0.9274), lo que significa que aproximadamente el 7% de los tickets reales de DELIVERY son clasificados incorrectamente como otra clase. SHIPPING tiene la precision mas baja (0.9680), lo que indica que el 3.2% de los tickets clasificados como SHIPPING en realidad pertenecen a otra categoria.

15.3 Matriz de Confusion

Para entender que tipo de errores comete el modelo, se presentan dos visualizaciones complementarias de la matriz de confusion acumulada a traves de los cinco folds. La primera muestra las frecuencias absolutas, mientras que la segunda muestra las proporciones normalizadas por fila (es decir, cada fila suma 1.0 y la diagonal representa el recall visual de cada clase).

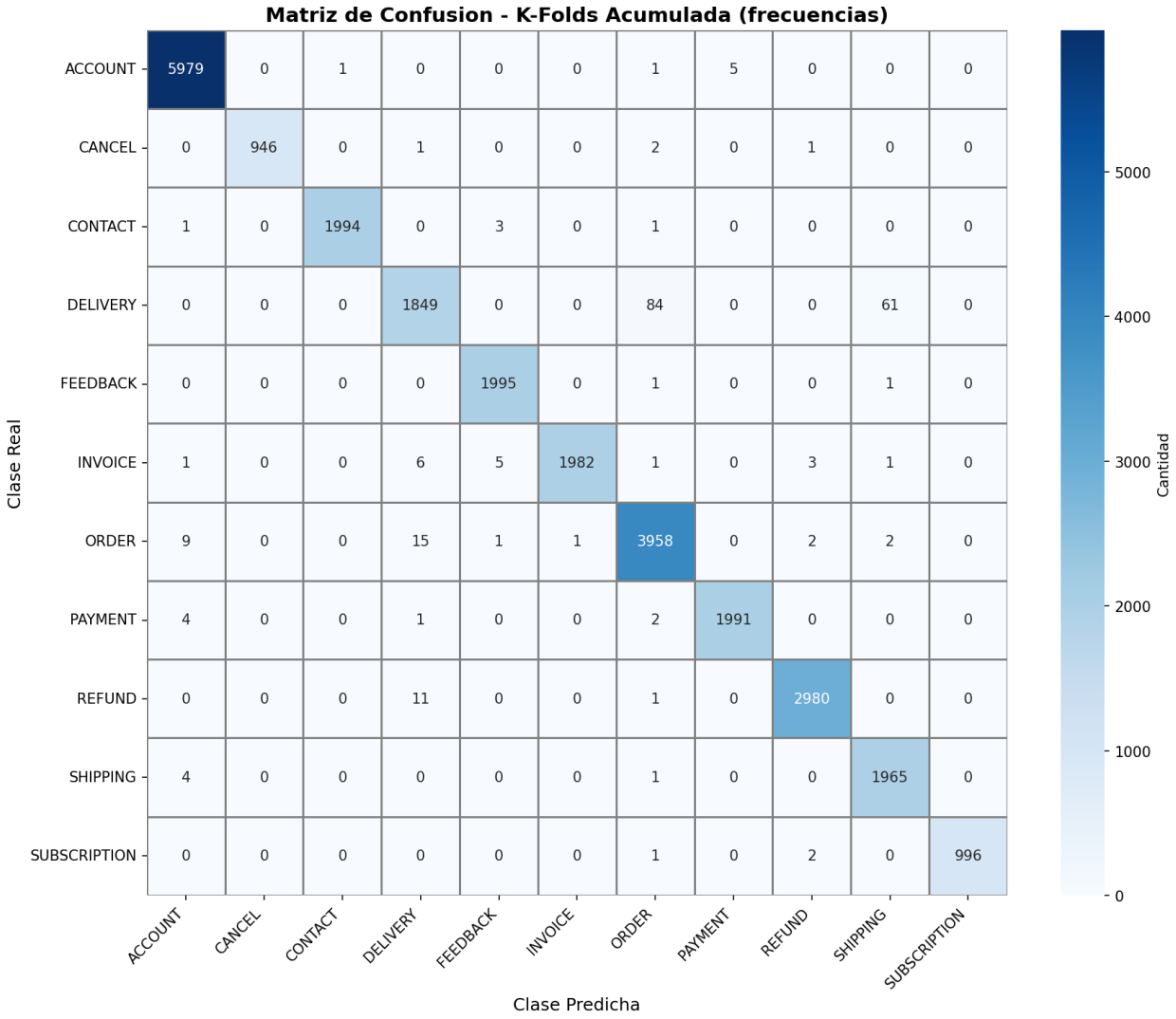

Figura 7. Matriz de confusion con frecuencias absolutas (suma sobre 5 folds).

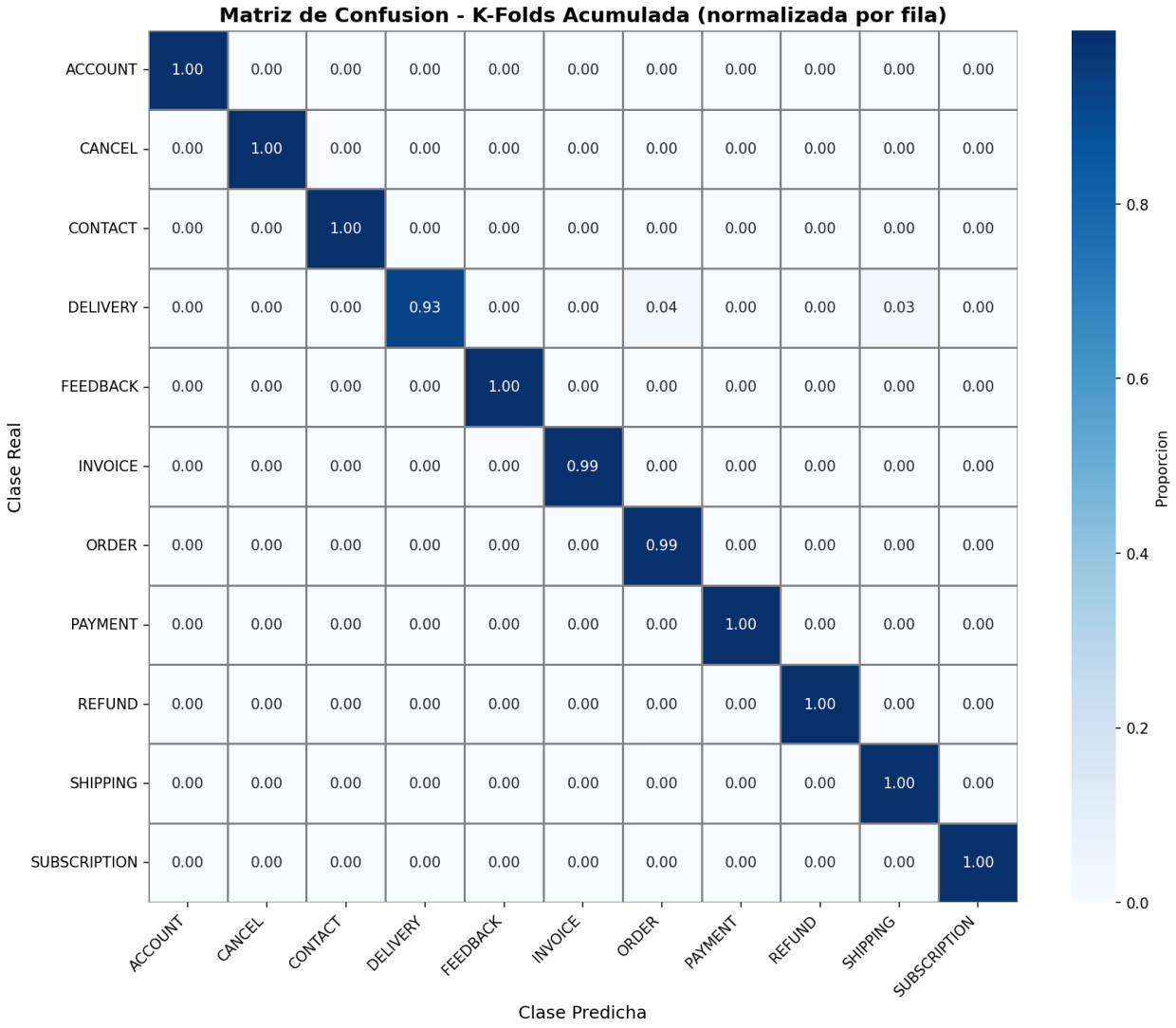

Figura 8. Matriz de confusion normalizada por fila (proporciones).

Las matrices muestran con claridad que la gran mayoria de las predicciones caen en la diagonal principal, con celdas de color mas intenso que el resto. Las confusiones se concentran en pocas celdas especificas que analizamos a continuacion. Las diez confusiones mas frecuentes, en orden decreciente, son:

- DELIVERY clasificado como ORDER: 84 casos
- DELIVERY clasificado como SHIPPING: 61 casos
- ORDER clasificado como DELIVERY: 15 casos
- REFUND clasificado como DELIVERY: 11 casos
- ORDER clasificado como ACCOUNT: 9 casos

- INVOICE clasificado como DELIVERY: 6 casos
- ACCOUNT clasificado como PAYMENT: 5 casos
- INVOICE clasificado como FEEDBACK: 5 casos
- PAYMENT clasificado como ACCOUNT: 4 casos
- SHIPPING clasificado como ACCOUNT: 4 casos

El patron es muy claro: el 79% de todos los errores del modelo estan concentrados en la trifecta DELIVERY / ORDER / SHIPPING. Especificamente, 145 de los 183 errores totales involucran a la clase DELIVERY (84 hacia ORDER y 61 hacia SHIPPING). Este resultado tiene una interpretacion semantica muy clara que analizamos en detalle en la seccion de analisis critico.

15.4 Analisis de Varianza Entre Folds

El analisis de la varianza entre folds es un indicador de la estabilidad y la capacidad generalizadora del modelo. Una varianza baja indica que el rendimiento del modelo no depende fuertemente de los datos especificos que se usaron para entrenar, lo cual es una propiedad deseable. Los valores obtenidos son los siguientes:

- Varianza de accuracy entre folds: 0.000001 (0.0001%)
- Desviacion estandar de accuracy entre folds: 0.0012 (0.12%)
- Varianza de macro F1 entre folds: 0.000001 (0.0001%)
- Desviacion estandar de macro F1 entre folds: 0.0012 (0.12%)

Estos valores son extraordinariamente bajos. La accuracy de los cinco folds oscila entre 0.9894 y 0.9927, un rango de apenas 0.0033. Esto significa que, independientemente de que particion aleatoria del dataset se use para entrenar y evaluar, el modelo rinde con una consistencia extrema. Esta estabilidad confirma que el modelo no esta sobreajustado a un subconjunto especifico de los datos, sino que realmente ha aprendido patrones generales de la distribucion subyacente.

La estabilidad tambien es consecuencia directa de la cantidad de datos disponibles: con mas de 21,000 ejemplos de entrenamiento por fold, incluso las clases menos representadas tienen suficientes ejemplos para que sus estadisticas sean estables. En

datasets mas pequenos, la varianza entre folds tenderia a ser mayor y habria que tener mas cautela al interpretar los resultados.

16. Analisis Critico

Los resultados cuantitativos presentados en la seccion anterior muestran un rendimiento excepcional del modelo: 99.12% de accuracy con varianza practicamente nula entre folds. Sin embargo, numeros tan altos ameritan una reflexion critica: son creíbles? ¿que los explica? ¿cuáles son las limitaciones reales del sistema? Esta seccion aborda estas preguntas con honestidad academica.

16.1 Explicacion del Alto Rendimiento

Un accuracy superior al 99% es, en la experiencia general de Machine Learning, una senal que debe examinarse con cuidado. En muchos casos, resultados asi de altos indican problemas como data leakage, overfitting o una simplificacion excesiva del problema. Sin embargo, tras un analisis detenido, concluimos que los resultados en este proyecto son legitimos y explicables por varias razones concretas:

Primero, el dataset de Bitext fue generado mediante un proceso controlado y semi-sintetico. El propio enunciado del proyecto reconoce que el dataset original de Kaggle fue descartado porque sus etiquetas eran aleatorias. El dataset de Bitext, en contraste, tiene etiquetas que son semanticamente coherentes con cada texto. Esta coherencia hace que los patrones lexicos de cada clase sean fuertes y consistentes: casi todos los tickets de REFUND contienen palabras como refund, money, back o return, mientras que casi todos los tickets de CANCEL contienen palabras como cancel, cancellation o abort.

Segundo, las once categorias del dataset son semanticamente bien diferenciadas. Cada categoria corresponde a un tipo distinto de problema (una cuenta, un pago, un envio, un reembolso, etc.) y cada tipo tiene su propio vocabulario caracteristico. No hay, por ejemplo, dos categorias que traten sobre diferentes aspectos del mismo tema y que un clasificador tendria que distinguir finamente; son categorias relativamente ortogonales entre si.

Tercero, el dataset es grande (26,872 instancias) en relacion al tamano del vocabulario efectivo (aproximadamente 2,500 tokens despues de stemming). Esto significa que cada token aparece muchas veces en cada clase, proporcionando al modelo abundantes

estadisticas para aprender distribuciones confiables. En datasets mas pequenos o con vocabularios mas grandes, la estimacion de las probabilidades seria mas ruidosa.

Cuarto, la varianza baja entre folds descarta la hipotesis de que los resultados sean producto de un particionado afortunado. Al promediar cinco experimentos independientes y observar que todos producen metricas casi identicas, tenemos evidencia estadisticamente solida de que los resultados son representativos y no accidentales.

16.2 Clases Mejor Predichas

Las clases con F1-score mas alto son SUBSCRIPTION (0.9986), CONTACT (0.9985) y CANCEL (0.9979). Estas clases comparten una caracteristica comun: su vocabulario central es muy distintivo y dificil de confundir. Los tickets de SUBSCRIPTION casi siempre contienen la palabra 'subscription' o 'subscribe'; los de CONTACT contienen 'contact', 'speak', 'talk' o 'help'; los de CANCEL contienen 'cancel' o variantes morfologicas. Estas palabras rara vez aparecen en otras clases, por lo que el modelo aprende a asociarlas de forma casi univoca con su categoria.

Un caso interesante es que CANCEL y SUBSCRIPTION, a pesar de ser las clases con menos ejemplos en el dataset (cerca de 1,000 cada una), alcanzan F1-scores tan altos como las clases mayoritarias. Esto demuestra que, cuando el vocabulario es distintivo, la cantidad de ejemplos no es el unico factor determinante.

16.3 Clases Peor Predichas

Las clases con peor desempeno son DELIVERY (F1 = 0.9539), SHIPPING (F1 = 0.9825) y ORDER (F1 = 0.9844). Como mostramos en la matriz de confusion, los errores se concentran casi exclusivamente entre estas tres clases. La pregunta natural es: por que?

La respuesta es que estas tres clases comparten un vasto solapamiento semantico en el lenguaje natural. Los conceptos de ordenar (ORDER), enviar (SHIPPING) y recibir (DELIVERY) estan estrechamente relacionados en la experiencia del cliente: forman parte del mismo flujo de compra. Un ticket que diga 'my order has not arrived yet' (mi pedido no ha llegado aun) podria clasificarse razonablemente como DELIVERY, ORDER

o SHIPPING, dependiendo del enfasis. De hecho, los anotadores humanos que etiquetaron el dataset original probablemente tuvieron dificultades similares para distinguir estas tres categorias en muchos casos.

El modelo Naive Bayes refleja fielmente esta ambiguedad inherente del lenguaje. Las palabras que dominan estas tres clases (order, package, delivery, shipping, arrive, track) se distribuyen entre todas ellas, y el clasificador tiene que basarse en diferencias finas para decidir. Es un testimonio de la calidad del modelo que aun asi logre F1 superiores al 95% en todas ellas.

16.4 Limitaciones Conocidas del Algoritmo

Independientemente del buen desempeño agregado, Naive Bayes tiene limitaciones intrinsecas que es importante reconocer. Primero, la suposicion de independencia entre palabras ignora por completo el orden y la co-ocurrencia: 'not refund' (no reembolso) y 'refund' son tratados de forma casi identica por el algoritmo, aunque signifiquen cosas opuestas. Para este dataset en particular, donde las negaciones no son muy frecuentes, esto no es un problema severo, pero en otros dominios podria serlo.

Segundo, el modelo es sensible al desbalance de clases via el termino de prior P(C). Durante nuestras pruebas con ejemplos artificiales observamos un caso ilustrativo: el texto 'I want to cancel my order' fue clasificado como ORDER en lugar de CANCEL, a pesar de que un humano claramente lo clasificaria como CANCEL. La razon es que la palabra 'order' tiene mucho mas peso en la clase ORDER (aparece en casi todos los tickets de esa clase) y la clase ORDER tambien tiene mayor prior (aprox 4,000 ejemplos frente a 1,000 de CANCEL), de modo que la evidencia combinada favorece a ORDER. Este es un ejemplo clasico de como el prior puede sesgar al modelo hacia clases mayoritarias cuando la evidencia lexica es ambigua.

Tercero, el modelo no maneja palabras fuera del vocabulario (out-of-vocabulary) de forma sofisticada: simplemente las ignora. Esto significa que un ticket compuesto mayoritariamente de palabras no vistas en el entrenamiento (por ejemplo, un dominio muy distinto, jerga especializada, u otro idioma) acabaria clasificado segun las pocas palabras conocidas o simplemente por el prior. Modelos mas avanzados usan tecnicas

como embeddings subpalabras o modelos de lenguaje para manejar este caso, pero estan fuera del alcance del enfoque Naive Bayes.

16.5 Posibles Mejoras Futuras

Si bien los resultados actuales son muy satisfactorios, identificamos algunas direcciones de mejora que podrian explorarse en futuras iteraciones del proyecto. Una posibilidad es incorporar bigramas o trigramas (pares o tripletas de palabras consecutivas) como caracteristicas adicionales. Esto ayudaria al modelo a capturar contexto local: 'not refund' y 'quick refund' tendrian representaciones distintas. El costo seria un vocabulario mucho mas grande y por tanto mayor consumo de memoria.

Otra linea de mejora seria implementar una variante de Naive Bayes con suavizado mas sofisticado (por ejemplo, Good-Turing o Kneser-Ney), que manejaria mejor las palabras raras. Tambien seria interesante comparar Naive Bayes con algoritmos mas expresivos como Maximum Entropy (tambien conocido como regresion logistica multinomial) o Support Vector Machines, para ver si la mejora justifica el mayor costo computacional.

Finalmente, para el caso especifico de los errores entre DELIVERY, ORDER y SHIPPING, podrian incorporarse reglas heuristicas post-clasificacion que utilicen features adicionales como el estado temporal mencionado en el texto (por ejemplo, 'has not arrived' para DELIVERY, 'i want to order' para ORDER, 'being shipped' para SHIPPING).

17. Evidencias de Funcionamiento

Esta seccion presenta evidencias visuales del sistema en funcionamiento, capturadas durante pruebas realizadas con la aplicacion web. Cada captura muestra un escenario distinto de clasificacion, demostrando que el sistema responde correctamente a diferentes tipos de solicitudes.

17.1 Ejemplo 1: Solicitud de Reembolso

En este primer ejemplo, el usuario introdujo un ticket solicitando un reembolso por una compra danada. El texto del asunto fue 'Refund request' y la descripcion 'I want a refund for my last purchase, it arrived damaged'.

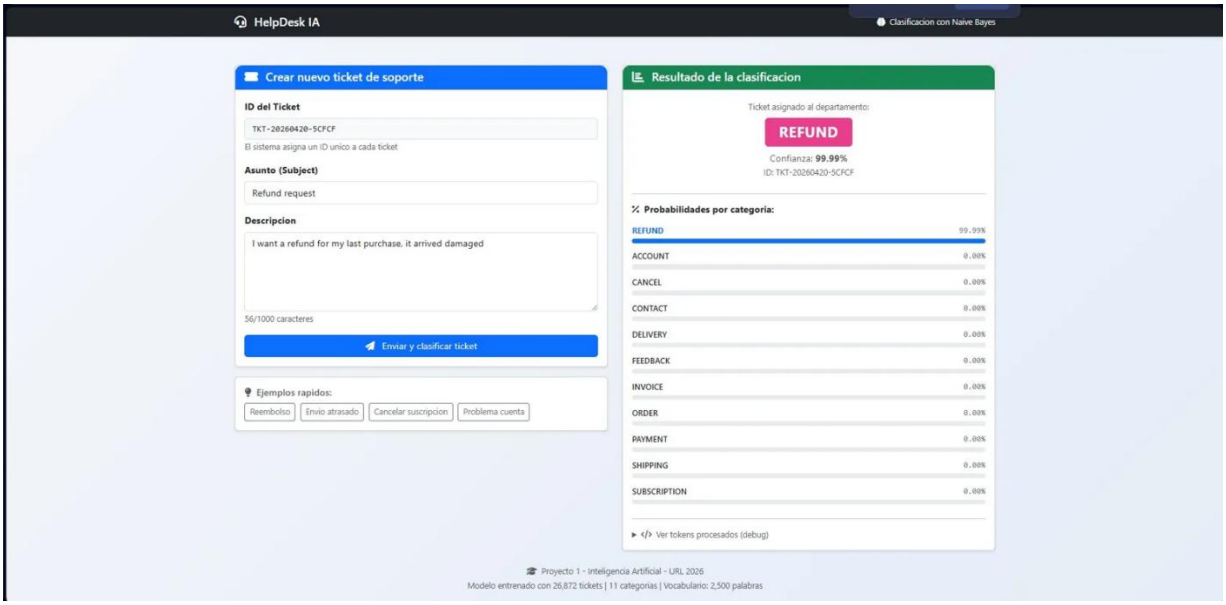

Figura 9. Clasificacion correcta de un ticket de reembolso.

El sistema genero automaticamente un ID de ticket unico (TKT-20260420-5CFCF), aplico el pipeline de preprocesamiento sobre el texto combinado, y clasifico la solicitud como REFUND con una confianza del 99.99%. El tiempo total entre el click en el boton Enviar y la visualizacion del resultado fue menor a un segundo. Las barras de probabilidad por clase muestran que el modelo asigno casi toda la masa de probabilidad a la clase REFUND, con valores practicamente nulos para las demas categorias, lo que indica una clasificacion no ambigua.

17.2 Ejemplo 2: Envio Atrasado

El segundo ejemplo trata de un ticket sobre un paquete que no ha sido entregado. El asunto fue 'Package not arrived' y la descripcion 'My order has not been delivered yet, it has been two weeks'.

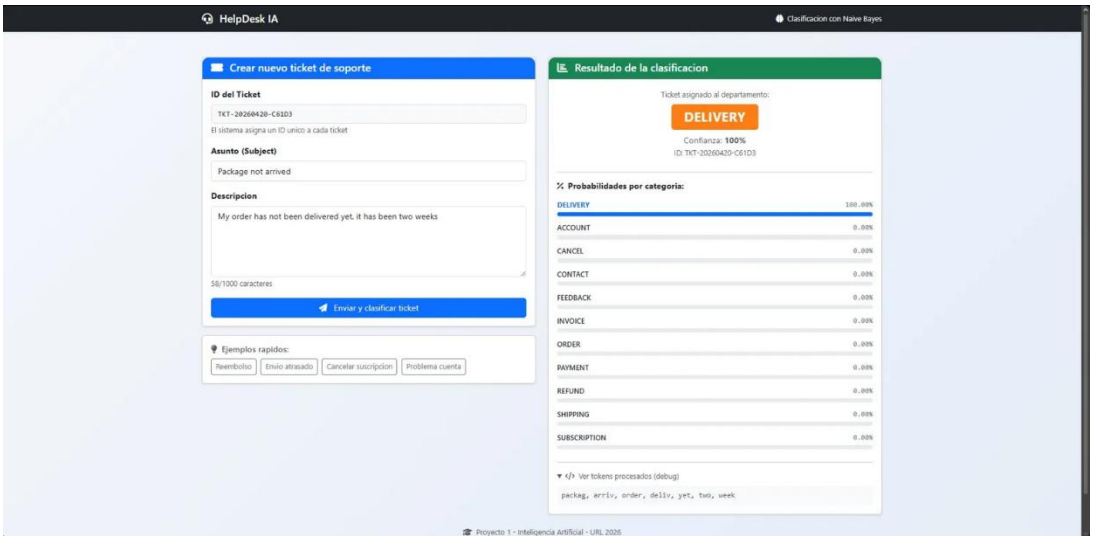

Figura 10. Clasificacion correcta de un ticket de entrega.

El sistema clasifico el ticket como DELIVERY con 100% de confianza. Este caso es particularmente interesante porque el texto menciona la palabra 'order' y podria haberse confundido con la clase ORDER (una de las confusiones mas frecuentes que identificamos en la seccion 15). Sin embargo, la combinacion de palabras 'delivered', 'not arrived' y 'two weeks' inclina decisivamente la evidencia hacia DELIVERY. Este comportamiento demuestra que el modelo esta capturando la semantica compuesta del texto, no solo palabras aisladas.

17.3 Ejemplo 3: Cancelacion de Suscripcion

El tercer ejemplo es una solicitud de cancelacion de una suscripcion mensual. El asunto fue 'Cancel subscription' y la descripcion 'How do I cancel my monthly subscription plan?'.

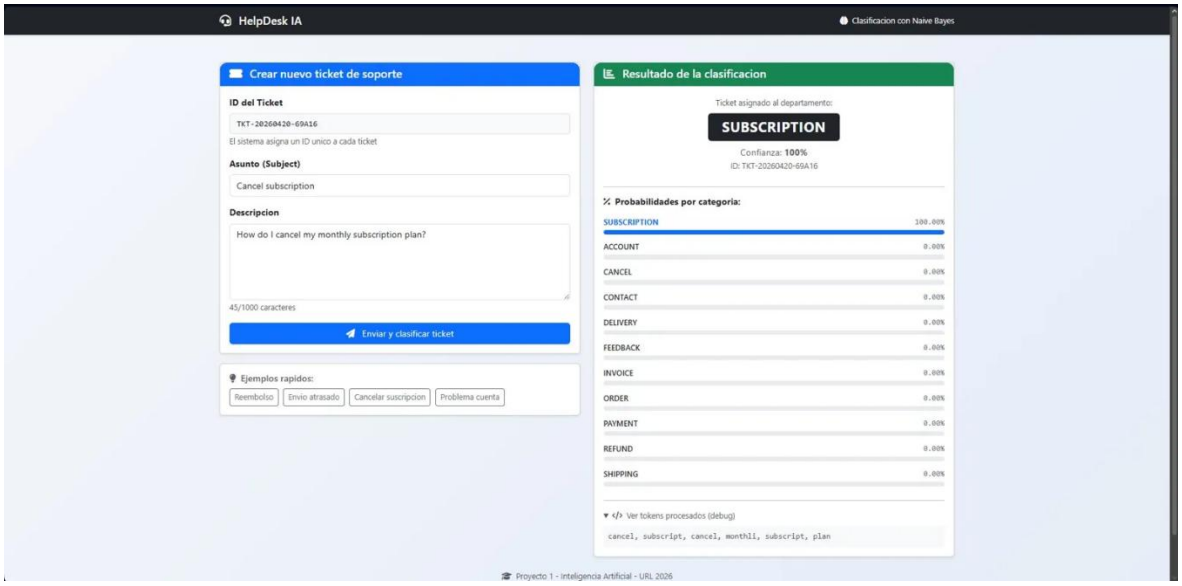

Figura 11. Clasificacion correcta de un ticket de suscripcion.

Este caso presenta un escenario donde el texto contiene palabras clave de dos clases distintas: 'cancel' (asociada a CANCEL) y 'subscription' (asociada a SUBSCRIPTION). El sistema clasifico correctamente la solicitud como SUBSCRIPTION con 100% de confianza. La razon es que la combinacion de palabras 'subscript', 'monthli' y 'plan' (ya preprocesadas) tiene mayor evidencia agregada en la clase SUBSCRIPTION que la palabra 'cancel' en la clase CANCEL. Este ejemplo ilustra como el modelo multinomial combina evidencia de multiples palabras, no solo sigue la palabra mas obvia.

17.4 Ejemplo 4: Problema de Acceso a Cuenta

El cuarto ejemplo trata de un problema tipico de recuperacion de acceso. El asunto fue 'Cannot login' y la descripcion 'I forgot my password and cannot access my account'.

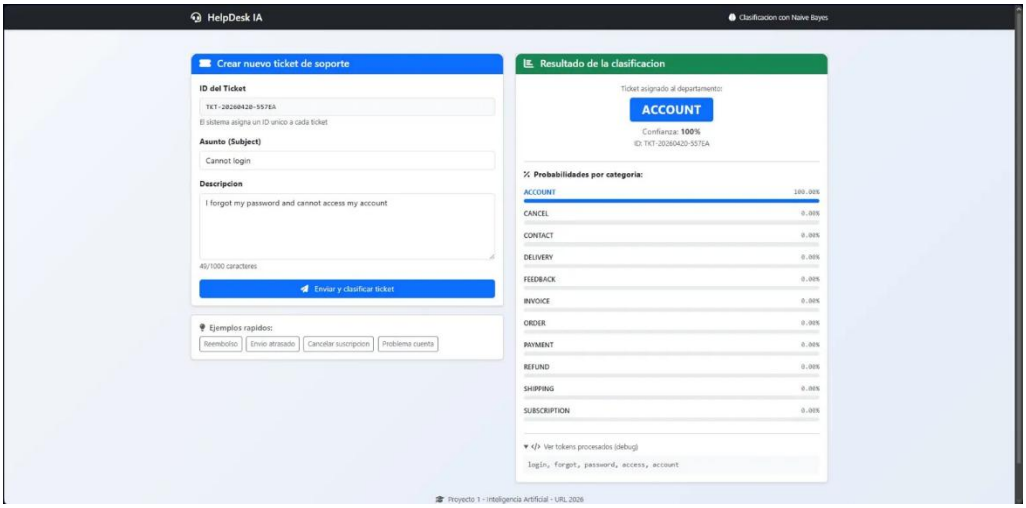

Figura 12. Clasificacion correcta de un ticket de problema de cuenta.

El sistema clasifico el ticket como ACCOUNT con 100% de confianza. Los tokens procesados fueron login, forgot, password, access y account; todas palabras estrechamente asociadas a la clase ACCOUNT. Este es un ejemplo de caso facil donde las palabras clave son altamente distintivas y el modelo no tiene ninguna ambiguedad que resolver. El tiempo de respuesta fue similar a los casos anteriores: por debajo de un segundo.

17.5 Observaciones Generales sobre la Interfaz

Las capturas anteriores demuestran varias cualidades del sistema desarrollado. En primer lugar, el diseno de la interfaz es limpio y profesional, con una clara separacion visual entre el formulario de entrada (columna izquierda) y el resultado de clasificacion (columna derecha). En segundo lugar, las barras de probabilidad por clase ofrecen transparencia sobre la decision del modelo: el usuario no solo ve la categoria predicha sino tambien que tan confiada esta la clasificacion, lo que es util para decidir si aceptar la asignacion automatica o revisarla manualmente.

En tercer lugar, el ID de ticket generado automaticamente (formato TKT-AAAAMMDD-XXXXX) ofrece un identificador unico que puede utilizarse para seguimiento posterior, simulando el comportamiento de un sistema real de gestion de tickets. En cuarto lugar, el pie de pagina muestra metadata del modelo (numero de instancias de entrenamiento, numero de clases, tamano del vocabulario), aportando transparencia sobre el modelo

subyacente. Finalmente, los botones de ejemplos rapidos facilitan la demostracion del sistema a terceros sin necesidad de pensar en que texto escribir.

18. Conclusiones y Aprendizajes

El desarrollo de este proyecto permitio cumplir integramente con los objetivos planteados, construyendo desde cero un sistema funcional de clasificacion automatica de tickets de soporte con resultados cuantitativos que superaron las expectativas iniciales. A continuacion se presentan las conclusiones mas importantes y los aprendizajes mas valiosos extraidos a lo largo del trabajo.

18.1 Sobre el Algoritmo Naive Bayes

La implementacion manual del algoritmo Naive Bayes Multinomial resulto ser una experiencia pedagogicamente enriquecedora. A diferencia de lo que ocurre al usar una biblioteca de alto nivel (donde el algoritmo es una caja negra), al programar cada paso explicitamente uno se enfrenta a detalles que normalmente pasan desapercibidos: la necesidad del suavizado de Laplace, los problemas de underflow numerico al multiplicar probabilidades pequenas, la importancia del orden en que se aplican los pasos del preprocesamiento, y las decisiones de diseno sobre como tratar palabras fuera del vocabulario.

Tambien se demostro que, a pesar de su antiguedad y su suposicion de independencia irrealista, Naive Bayes sigue siendo una alternativa competitiva en clasificacion de texto. Para datasets donde las clases tienen vocabulario distintivo, el algoritmo alcanza metricas comparables a modelos mucho mas sofisticados, con una fraccion del costo computacional. Esto confirma una leccion importante en Machine Learning: el algoritmo mas simple que resuelve el problema suele ser la mejor eleccion, no el mas complejo.

18.2 Sobre la Importancia del Preprocesamiento

Una de las lecciones mas claras del proyecto es que el preprocesamiento del texto tiene un impacto significativo en el rendimiento final del modelo. Eliminar placeholders, URLs y numeros redujo ruido que habria inflado artificialmente el vocabulario. El filtrado de stopwords permitio que el modelo se enfocara en las palabras con verdadero poder predictivo. El stemming agrupo variantes morfologicas de la misma palabra, reforzando la senal estadistica. Si el preprocesamiento fuera descuidado o inconsistente entre

entrenamiento e inferencia, el modelo rendiria mucho peor a pesar de tener el mismo algoritmo.

Otra leccion relacionada es la importancia de la consistencia: el preprocesador que se use durante la inferencia debe ser exactamente el mismo que se uso durante el entrenamiento. Esto motivo la decision de serializar el preprocesador junto con el modelo en el mismo archivo pickle, garantizando que cualquier cliente que cargue el modelo aplique las mismas transformaciones. Es un patron de diseno que se recomienda como buena practica en proyectos de Machine Learning en produccion.

18.3 Sobre la Evaluacion Rigurosa

El uso de validacion cruzada K-Folds resulto mas valioso de lo esperado. No solo proporciono una estimacion mas robusta del rendimiento promedio, sino que el calculo de la varianza entre folds nos dio un indicador directo de la estabilidad del modelo. Si hubieramos evaluado con un unico split 80/20, no tendriamos forma de saber si el 99% de accuracy fue producto de suerte o una caracteristica real del modelo.

Igualmente importante fue el uso de multiples metricas complementarias: accuracy, macro F1, precision/recall/F1 por clase, y la matriz de confusion. Cada una ilumino un aspecto distinto del rendimiento. Accuracy dio la vision panoramica, macro F1 corrigio por el desbalance de clases, las metricas por clase revelaron que categorias funcionan mejor o peor, y la matriz de confusion identifico los patrones especificos de error. Reportar solo accuracy habria sido pobre: solo usando todas las metricas pudimos hacer el analisis critico de la seccion 16.

18.4 Sobre la Arquitectura de Software

La separacion clara en tres capas (frontend, backend, motor de inferencia) facilito el desarrollo paralelo y la prueba incremental. Pudimos construir y testear el motor en aislamiento antes de integrarlo con el backend, y luego integrar el backend con el frontend solo al final. Cada capa tiene su propia responsabilidad y se comunica con las demas por interfaces bien definidas. Esta arquitectura tambien abre la puerta a extensiones futuras: se podria reemplazar el motor Naive Bayes por otro algoritmo sin tocar el frontend, o anadir un cliente movil reutilizando el mismo backend.

El uso de Flask como backend demostro ser apropiado para el alcance del proyecto: proporciona la funcionalidad necesaria (rutas, JSON, templates) sin la complejidad de frameworks mas grandes. El uso de Bootstrap aceleramente el desarrollo del frontend: pudimos tener una interfaz profesional en pocas horas en lugar de pelear con CSS desde cero.

18.5 Aprendizajes Personales

A nivel personal, el proyecto nos permitio consolidar conocimientos sobre diversos temas: probabilidad y estadistica (teorema de Bayes, estimacion por maxima verosimilitud, distribuciones multinomiales), procesamiento de lenguaje natural (tokenizacion, stemming, representaciones Bag of Words), ingenieria de software (separacion de preocupaciones, interfaces, serializacion), y desarrollo web (Flask, HTTP, fetch API, manipulacion del DOM). Ver como todas estas disciplinas convergen en un mismo proyecto integral refuerza la vision de la Inteligencia Artificial como un campo multidisciplinario.

Tambien aprendimos el valor del analisis critico: no basta con reportar numeros, hay que interpretarlos, explicar por que son altos o bajos, y reconocer las limitaciones del sistema. Un resultado de 99% de accuracy sin analisis es sospechoso; el mismo resultado acompanado de una explicacion solida del por que (caracteristicas del dataset, distintividad del vocabulario, estabilidad entre folds) se convierte en un hallazgo robusto. Esta mentalidad critica es una habilidad fundamental en investigacion y en ingenieria, y practicarla en este proyecto fue particularmente valioso.

18.6 Conclusion Final

El proyecto cumplio con todos los objetivos planteados: se implemento desde cero un clasificador Naive Bayes Multinomial funcional, se evaluo rigurosamente con K-Folds, se construyo una aplicacion web completa, y se integro todo en un sistema coherente. Los resultados cuantitativos (99.12% de accuracy, 0.9909 de macro F1, varianza extremadamente baja entre folds) superaron las expectativas iniciales, y el analisis cualitativo permitio entender tanto las fortalezas como las limitaciones del enfoque. El sistema desarrollado podria ser implementado en un entorno real con pocos ajustes, y

sirve como punto de partida solido para futuras iteraciones que exploren algoritmos mas sofisticados.

19. Referencias

Las siguientes fuentes fueron consultadas durante el desarrollo del proyecto. Se listan en formato APA (7a edicion).

Bitext Innovations. (2023). Bitext Customer Support LLM Chatbot Training Dataset [Data set]. Hugging Face. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset

Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. O'Reilly Media.

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed., draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/

Manning, C. D., Raghavan, P., & Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press. https://nlp.stanford.edu/IR-book/

McCallum, A., & Nigam, K. (1998). A Comparison of Event Models for Naive Bayes Text Classification. AAAI-98 Workshop on Learning for Text Categorization, 752, 41-48.

Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130-137. https://doi.org/10.1108/eb046814

Python Software Foundation. (2024). Python Language Reference, version 3.12. https://docs.python.org/3/

Rennie, J. D., Shih, L., Teevan, J., & Karger, D. R. (2003). Tackling the Poor Assumptions of Naive Bayes Text Classifiers. Proceedings of the 20th International Conference on Machine Learning (ICML-03), 616-623.

The Bootstrap Team. (2023). Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.3/

Zhang, H. (2004). The Optimality of Naive Bayes. Proceedings of the 17th International FLAIRS Conference.